PH B-spline family: Technical Implementation Specification

Dynamic planar Pythagorean-hodograph B-splines with verified distance-domain access

Prepared from the cubic-ph-spline 1.1.0 reference implementation

2026-08-11

Contents

Document status

Status: normative implementation specification, revision 0.5.

Target: the repository PH B-spline family reference implementation, generalized from immutable cubic PH segments to mutable, variable-order planar PH B-splines. This revision incorporates the minimum-degree basis correction and the periodic/antiperiodic closed-preimage seam correction.

Reviewed baseline: repository `main` as retrieved on 2026-08-05; package metadata reports version 1.1.0. The retrieved test suite completed with **491 passed** under the review environment. The baseline package documents normalized construction, independently verified nonlinear solves, exact polynomial arc length, cancellation-resistant cubic inversion, and approximately 5-6 microseconds per scalar random `point_at_length` query for 100 through 10,000 segments on its reported benchmark platform.

Current verification: the combined repository suite completed with **900 passed** after this reconstruction audit and the revision-0.5 exact-offset implementation. Three additional parametrizations skip by topology (a closed-seam check on open cases); no test is skipped for a missing feature.

This document is normative for reconstruction of the reference mathematical core. Sections explicitly marked **future extension** are design directions, not claims about the current implementation. Where an earlier revision and the

reference code disagreed, this revision describes the independently tested code path; silent substitution of a different degree, seam topology, solver objective, or indexing structure is nonconforming.

Revision 0.5 makes exact NURBS offsets a required production feature. The construction, the shared read-only `NURBSHandle`, and the acceptance tests of Sections 6.9, 15.6, 22.11, and 23 are implemented in `ph_spline/nurbs.py` plus the family `offset()` methods and pass in the verification count above. The offset acceptance oracle uses exact rational arithmetic, which exceeds the required 100-decimal-digit precision. Earlier test counts do not establish offset conformance.

The terms **MUST**, **MUST NOT**, **REQUIRED**, **SHALL**, **SHALL NOT**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **MAY**, and **OPTIONAL** are to be interpreted as requirement levels.

Executive specification

`PHBSplineOpen` and `PHBSplineClosed` SHALL be mutable planar point-interpolating splines with the following principal properties. `PHBSpline` SHALL be their non-instantiable abstract family base:

1. It accepts an ordered sequence of finite planar points in arbitrary geometric configuration: convex, nonconvex, inflectional, self-intersecting, looping, backtracking, or containing nonconsecutive repeated points. Consecutive coincident points are rejected by default because they do not define a nonzero interpolation span.
2. It represents a piecewise polynomial Pythagorean-hodograph curve using a complex polynomial B-spline preimage. Its speed and cumulative arc length are piecewise polynomials constructed analytically, without numerical quadrature.
3. It supports any requested finite continuity orders `g_order`, `c_order`, and `curvature_order`, subject to configured degree and resource limits. If none are supplied, the default guarantee is `G2`.
4. It supports exact interpolation-point move, insertion, and deletion as atomic local patch operations. Geometry compilation is local and unchanged exterior span kernels are structurally shared. The reference implementation still performs whole-object validation, array assembly, and flat-prefix publication, so total edit latency is currently $O(N)$ even though the nonlinear solve and span recompilation are bounded.
5. They provide an API adapted from `CubicPHSplineOpen`: `point`, `tangent`, `normal`, `principal_normal`, `signed_curvature`, `curvature_vector`, `arc_length`, `parameter_at_length`, and `point_at_length`, plus arbitrary-order derivative-vector and curvature-vector queries, batch queries, and dynamic-edit APIs.
6. Random distance access uses a compensated flat prefix array, binary search, a per-span parameter/length LUT, and bracketed Newton correction with bisection fallback. The result is accepted only after a forward or reverse arc-length residual test near binary64 precision.
7. Every constructor and edit operation is transactional. Solver success flags are never sufficient. Interpolation, continuity, PH reconstruction, regularity, arc-length monotonicity, and inverse-kernel postconditions are independently verified before commit.
8. The internal curve, hodograph, speed, arc-length polynomial, tangent, normal, curvature, and PH offset data are constructively derived from the preimage coefficients. The interpolation coefficients themselves are generally obtained by a deterministic nonlinear solve, not by a universal symbolic formula.
9. Every finite signed parallel offset whose binary64 coordinates are representable is constructed exactly as an immutable NURBS. The construction uses polynomial coefficient products from the PH representation. It does not sample or fit an approximate curve.

The implementation SHALL NOT claim unconditional existence of a regular fixed-degree PH interpolant for every possible input and every requested order. Its contract is:

> For every valid input, construction either returns a spline satisfying all declared guarantees and verification bounds, or raises a typed exception containing structured diagnostic information. It never returns an unverified or silently downgraded curve.

1. Scope

1.1 In scope

The first production version SHALL implement:

- open and closed planar splines;

- ordered point interpolation;
- variable finite continuity requirements;
- a polynomial PH representation;
- dynamic point move, insertion, deletion, append, and prepend;
- local transactional reconstruction;
- exact analytic forward arc length;
- efficient scalar and vectorized distance-domain evaluation;
- arbitrary-order spline-parameter derivative vectors and curvature-vector derivatives;
- deterministic binary64 numerics with explicit failure modes;
- immutable query snapshots;
- exact signed parallel offsets returned as immutable read-only NURBS handles;
- typed exceptions and diagnostics;
- persistence of the authoritative interpolation data and verified compiled representation;
- test and benchmark infrastructure capable of detecting numerical and asymptotic regressions.

1.2 Explicitly out of scope for version 1

The following are not required for the first implementation:

- spatial or quaternion PH splines;
- rational PH input curves;
- PH surfaces;
- global minimum-bending-energy guarantees;
- universal loop-free or self-intersection-free interpolation;
- automatic obstacle avoidance;
- exact symbolic inverse of a generic high-degree arc polynomial;
- a proof that every arbitrary point sequence admits a regular solution under a fixed hidden-span cap;
- GPU kernels;
- exact arithmetic construction;
- a public control polygon for the source PH B-spline with the same semantics as a NURBS control polygon; the exact offset's read-only NURBS controls are required by Section 6.9.

These omissions MUST be documented in the public package.

2. Baseline compatibility and intentional differences

2.1 API compatibility objective

The following scalar methods SHALL retain the baseline naming and result conventions:

```
point(u) -> np.ndarray          # shape (2,), dtype float64
tangent(u) -> np.ndarray        # unit vector
normal(u, side="left") -> np.ndarray
principal_normal(u) -> np.ndarray
signed_curvature(u) -> float
curvature_vector(u) -> np.ndarray
arc_length(u) -> float
parameter_at_length(s) -> float
point_at_length(s) -> np.ndarray
offset(distance) -> NURBSHandle
```

The baseline `curvature_vector(u)` call SHALL remain source-compatible and mean order zero. Both concrete PH B-spline classes SHALL additionally expose named arbitrary-order vector queries:

```
derivative(u, order=1, *, side="auto") -> np.ndarray
curvature_vector(u, order=0, *, side="auto") -> np.ndarray
```

These methods are not numerical-differentiation conveniences. They are required geometry kernels with elementary coefficient formulas, scale-aware error control, and the same strict failure behavior as distance and curvature evaluation.

Scalar arguments SHALL accept Python and NumPy real scalars, reject booleans, arrays, sequences, NaN, and infinities, and apply only a documented few-ULP endpoint clamp. Array input belongs to explicit batch methods.

2.2 Baseline design principles that SHALL be retained

The new package SHALL retain these principles from `cubic-ph-spline`:

- normalized construction rather than raw-coordinate nonlinear solving;
- deterministic initializers and bounded iteration counts;
- independent acceptance checks after every nonlinear solve;
- no generic numerical quadrature for arc length;
- no generic geometric search for `point_at_length`;
- cancellation-resistant endpoint handling;
- explicit regularity margins;
- structured exceptions carrying index, quantity, measured value, and required bound;
- warnings treated as test failures;
- extensive adversarial scale, curvature, chord-ratio, and round-trip tests;
- immutable compiled segment kernels even when the owning spline is mutable.

2.3 Intentional differences

The PH B-spline family differs from `CubicPHSplineOpen` in the following mandatory ways:

Concern	`CubicPHSplineOpen` baseline	PH B-spline requirement
Primitive degree	Cubic PH segments	Degree deduced from continuity request; default quintic PH
Arc inverse	Elementary monotone cubic inverse	Cached monotone inverse plus safeguarded polynomial correction
Object mutability	Immutable	Mutable, transactional, with immutable snapshots
Metric index	Flat compensated prefix array	Flat compensated prefix array in the reference profile; an augmented tree is a future extension
Nonconvex handling	Specialized cubic preprocessing	No convexity admissibility restriction; generic regular PH solve
Continuity	G2 on convex runs, G1 at selected transitions	User-selected finite G, C, and curvature continuity, verified at every join
Geometry queries	Tangent and order-zero curvature vector	Arbitrary-order spline-parameter derivative vectors and curvature-vector derivatives
Local editing	Not supported	Move/insert/delete with bounded local patch mode
Representation	Cubic Bezier plus linear preimage	Variable-order B-spline preimage plus per-span Bernstein kernels
Exact offset output	Piecewise rational quintic NURBS	Piecewise rational degree $4m + 1$ NURBS for preimage degree m

2.4 Repository source audit and reuse plan

The implementation plan is based on a direct review of the repository source and tests, not only its README. The following mapping SHALL guide reuse and replacement:

Baseline file	Observed responsibility	Requirement for `PHBSpline`
`ph_spline/cubic.py`	Immutable public API, scalar validation, global parameter dispatch, prefix-length lookup, construction post-verification	Retain scalar semantics and verification style; use immutable span kernels and atomic state publication; keep flat prefixes in the reference profile
`segment.py`	Dual cubic Bezier and linear complex-preimage storage; point, tangent, speed, curvature, and local inverse methods	Generalize to variable-degree immutable `SpanKernel` with Bernstein preimage, exact speed/arc controls, and compiled numerical inverse
`arclength.py`	FMA Horner forms, cancellation-aware cubic arc evaluation, scaled hyperbolic/Cardano estimate, endpoint-reversed inversion, bounded safeguarded Newton, explicit residual gate	Retain forward/reverse evaluation, fixed iteration bounds, residual verification, and endpoint exactness; replace the cubic estimator by the LUT seed and generic monotone polynomial correction
`construction.py`	Input validation, normalization, geometric planning, segment assembly	Retain validation and normalized solving; replace convex-run/cubic-specific planning with a minimum-degree simple-knot preimage and exact displacement constraints
`nonlinear.py`	Deterministic bounded nonlinear solve with structured fallback and independent acceptance	Retain determinism, analytic/complex-step verification, damping, and hard bounds; generalize to banded equality-constrained PH displacement solves
`exceptions.py`	Value/runtime dual inheritance and structured diagnostics	Preserve the pattern, expand fields for point IDs, span IDs, edit operation, and patch
`tests/`	Arc inversion, extreme scale, frames, G2, nonconvex data, invariants, straight cases, and validation	Port all behavioral intents and add variable-order, dynamic-edit, flat-prefix, closed-monodromy, and scale tests

The package top-level `ph_spline.__init__` SHALL export `PHSpline`, the abstract family bases `CubicPHSpline` and `PHBSpline`, and all four sibling topology classes `CubicPHSplineOpen`, `CubicPHSplineClosed`, `PHBSplineOpen`, `PHBSplineClosed`, and the common `NURBSHandle`. No open/closed pair and no cubic/B-spline pair may inherit from one another; concrete classes inherit only their own family base.

3. Mathematical model

3.1 Complex planar representation

Represent a planar point by

$$z(t) = x(t) + iy(t).$$

A planar polynomial PH curve is generated by a complex polynomial or spline preimage $w(t)$:

$$z'(t) = w(t)^2.$$

Its speed is

$$\sigma(t) = |z'(t)| = |w(t)|^2 = w(t)\overline{w(t)},$$

and its arc length is

$$S(t) = \int \sigma(t)\, dt.$$

If w is a degree-m polynomial on a knot span, then:

$$\deg z = 2m + 1, \qquad \deg \sigma = 2m, \qquad \deg S = 2m + 1.$$

3.2 Logical B-spline preimage

The logical preimage is

$$w(t) = \sum_j c_j N_{j,m}(t), \qquad c_j \in \mathbb{C},$$

on a nondecreasing knot sequence. The implementation MAY store the curve in segmented Bezier-extracted form, but the authoritative pieces MUST be compatible with a B-spline preimage and MUST pass left/right preimage-jet verification at every knot.

With degree m and interior knot multiplicity ν, generically:

$$w \in C^{m-\nu}, \qquad z \in C^{m-\nu+1}, \qquad \kappa \in C^{m-\nu-1}.$$

The default construction SHALL use simple interior knots, $\nu = 1$, unless a specific lower continuity is explicitly requested as part of a future extension. Thus:

$$w \in C^{m-1}, \qquad z \in C^{m}, \qquad \kappa \in C^{m-2}.$$

For N open user intervals split once at their midpoints, the reference basis has $2N$ polynomial spans and $2N + m$ complex B-spline controls. For a closed curve with N cyclic user intervals, it has $2N$ spans and $2N$ independent complex controls, with the remaining extended controls supplied by the seam law below. These counts, the knot multiplicities, and the extraction matrices are part of the basis definition; they MUST NOT be changed merely to obtain more nonlinear-solver freedom.

3.3 Closed preimage monodromy

The curve is closed when its hodograph and integrated displacement close. It does **not** follow that the square-root preimage is periodic. The closed preimage SHALL carry a monodromy sign

$$\eta \in \{+1, -1\}, \qquad w^{(q)}(t + T) = \eta\, w^{(q)}(t), \quad q = 0, \dots, m - 1.$$

Thus $\eta = +1$ is periodic and $\eta = -1$ is antiperiodic. Both produce a periodic hodograph because $(\eta w)^2 = w^2$. More generally, if the tangent turning number is n_T, a continuous square-root lift has $\eta = (-1)^{n_T}$. In particular, every regular simple closed planar curve has $n_T = \pm 1$ and therefore requires the antiperiodic lift.

Let $n = 2N$ be the number of compiled spans and base controls. Closed extended controls SHALL obey the twisted cyclic extension

$$c_{j+qn} = \eta^q c_j, \qquad 0 \le j < n, \quad q \in \mathbb{Z}.$$

Bezier extraction across the seam SHALL apply this sign before wrapped columns are collapsed modulo n. This rule also covers $m \geq n$, where one extraction stencil may wrap more than once. A plain modulo operation is correct only when $\eta = +1$.

The seam verifier SHALL compare physical preimage jets with the same sign:

$$h_{n-1}^{-q} \frac{d^q w_{n-1}}{d\nu^q}(1) = \eta\, h_0^{-q} \frac{d^q w_0}{d\nu^q}(0), \qquad q = 0, \dots, m-1.$$

Ordinary joins use $\eta = +1$. Curve, tangent, speed, curvature, and all derived curve jets remain periodic at an antiperiodic seam; the sign belongs to the non-observable square-root gauge, not to the geometry.

3.4 Local span coordinate

For a knot span $[\tau_i, \tau_{i+1}]$, define

$$h_i = \tau_{i+1} - \tau_i > 0, \qquad u = \frac{t - \tau_i}{h_i} \in [0, 1].$$

Let the Bezier-extracted preimage be

$$w_i(\nu) = \sum_{a=0}^{m} b_{i,a} B_a^m(\nu).$$

Then

$$\frac{dz}{d\nu} = h_i w_i(\nu)^2, \qquad \frac{ds}{d\nu} = h_i |w_i(\nu)|^2.$$

All runtime span kernels SHALL use $\nu \in [0, 1]$. No evaluator SHALL form a high-degree global power polynomial in an unscaled global parameter.

3.5 Bernstein product formula

For degree-m Bernstein coefficients b_a, the degree-$2m$ hodograph coefficients are

$$q_k = h_i \sum_{a+b=k} \frac{\binom{m}{a}\binom{m}{b}}{\binom{2m}{k}} b_a b_b, \qquad k = 0, \dots, 2m.$$

The speed coefficients are

$$r_k = h_i \sum_{a+b=k} \frac{\binom{m}{a}\binom{m}{b}}{\binom{2m}{k}} b_a \overline{b_b}.$$

The imaginary part of each computed r_k SHALL be checked against a rounding bound and discarded only after that check. Silent conversion of a materially complex speed coefficient to real is forbidden.

3.6 Bernstein antiderivative

For a degree-d Bernstein polynomial

$$f(\nu) = \sum_{k=0}^{d} a_k B_k^d(\nu),$$

an antiderivative with $F(0) = 0$ has coefficients

$$A_0 = 0, \qquad A_{k+1} = A_k + \frac{a_k}{d+1}, \qquad k = 0, \dots, d.$$

This formula SHALL generate both the position and arc-length coefficients. Numerical quadrature SHALL NOT be used to produce authoritative span lengths.

4. Continuity contract and degree selection

4.1 Constructor continuity parameters

The constructor SHALL expose:

```
g_order: int | None = None
c_order: int | None = None
curvature_order: int | None = None
```

Semantics:

- `g_order=r` requests oriented geometric continuity G^r.
- `c_order=r` requests parametric continuity C^r in the package global parameter.
- `curvature_order=k` requests signed curvature continuity $\kappa \in C^k$ with derivatives taken with respect to arc length.
- `None` means that continuity type is not independently requested.
- If all three are `None`, the constructor SHALL set `g_order=2`.
- Boolean values are invalid despite `bool` being an `int` subclass.
- Valid orders are nonnegative integers subject to the configured resource limit.

4.2 Degree rule

Define

$$r_* = \max\left(2,\ g_{\text{req}},\ c_{\text{req}},\ k_{\text{req}} + 2\right),$$

where a missing request contributes zero. The base preimage degree SHALL be

$$m = r_*,$$

and the curve degree SHALL be

$$p = 2m + 1.$$

The minimum $m = 2$ is intentional. It gives quintic PH spans and avoids the fixed-curvature-sign restriction of regular planar PH cubics.

The implementation SHALL use exactly $m = r_*$. Shape freedom and numerical regularity SHALL be obtained by inserting simple knots, not by increasing the polynomial degree beyond the continuity requirement. The object MUST report both the selected preimage degree and curve degree.

This rule follows from the simple-knot continuity chain and is not a tuning heuristic:

- $w \in C^{m-1}$ implies $z' = w^2 \in C^{m-1}$ and hence $z \in C^m$;
- a regular C^{m-1} preimage gives $\kappa \in C^{m-2}$;
- oriented G^r requires tangent agreement and arc-length curvature derivatives through order $r-2$, so $m \geq r$ suffices;
- parametric C^r requires $m \geq r$;
- curvature C^k requires $m \geq k + 2$.

Therefore the maximum in r_* is the least preimage degree that satisfies all three requested guarantees under the reference simple-knot construction, subject to the intentional floor $m \geq 2$. A higher degree is not needed to impose the constraints. The apparently spare coefficients are shape and interpolation degrees of freedom; they SHALL be supplied by the midpoint knot topology and selected by the deterministic guide projection. Higher preimage derivatives SHALL NOT be arbitrarily set to zero as a substitute for choosing the correct degree.

The reference construction inserts one simple knot at the parameter midpoint of every user interval. Consequently, each user interval contains two compiled polynomial spans. A simple knot preserves generic C^{m-1} preimage continuity while supplying one additional complex control degree of freedom per interval; exact PH displacement constraints retain interpolation at the user knots.

Counterexample (over-degree construction). Selecting $m = 2r+1$ for a G^r request does not create a stronger requirement; it introduces unconstrained high-order shape modes, increases span degree from $2r + 1$ to $4r + 3$, worsens endpoint-difference conditioning, and makes every query more expensive. Setting those extra modes to zero merely hides the incorrect basis choice. Conformance tests require `preimage_degree == r_*`.

Examples:

Requested guarantee	Preimage degree m	Curve degree $2m+1$	Generic simple-knot result
default `G2`	2	5	at least C2, curvature C0
`C3`	3	7	C3, curvature C1
`G4`	4	9	at least C4, curvature C2
curvature `C2`	4	9	C4, curvature C2
`C5`, curvature `C3`	5	11	C5, curvature C3

4.3 Geometric continuity verification

For a regular oriented planar curve parameterized by arc length s:

$$T_s = i\kappa T.$$

The verifier SHALL treat G^r as matching:

$$T_- = T_+,$$

and, for $r \geq 2$,

$$\frac{d^j \kappa_-}{ds^j} = \frac{d^j \kappa_+}{ds^j}, \qquad j = 0, \ldots, r-2.$$

If stronger parametric C^r continuity is produced by the B-spline construction, that is acceptable and SHALL be reported as an actual guarantee.

4.4 Parametric continuity verification

For `c_order=r`, the implementation SHALL verify physical left/right derivatives with respect to the global parameter through order r:

$$\frac{d^j z_-}{dt^j} = \frac{d^j z_+}{dt^j}, \qquad j = 0, \ldots, r.$$

The verifier SHALL account for span widths h_i; comparing unscaled local derivatives is incorrect.

4.5 Curvature derivatives

Curvature derivatives SHALL be defined with respect to arc length unless explicitly requested otherwise:

$$D_s = \frac{1}{\sigma(t)} D_t.$$

The implementation SHALL compute curvature jets using formal truncated power-series arithmetic or an algebraically equivalent recurrence. Finite differences are forbidden for continuity acceptance.

5. Input domain and geometry policy

5.1 Accepted point configurations

The input is an **ordered** point sequence, not an unordered point cloud. Subject to finite representability and nonzero consecutive chords, the implementation SHALL accept attempts to construct splines through:

- convex and nonconvex polylines;
- any number of inflections;
- self-intersecting polylines;
- curves that must loop to satisfy continuity;
- exact or near direction reversals;
- alternating very short and very long chords;
- nonconsecutive repeated points;
- open near-closed paths;
- closed paths.

No convexity, simplicity, orientation, monotone-turn, or star-shaped predicate may be a constructor precondition.

5.2 Invalid point data

The constructor SHALL reject:

- fewer than two points for an open curve;
- fewer than three distinct cyclic anchors for a closed curve;
- malformed arrays or elements;
- booleans as coordinates;
- NaN or infinite coordinates;
- consecutive points that compare equal after conversion to binary64;
- a chord whose physical displacement or length cannot be represented reliably under the numerical policy;
- an input whose requested continuity order exceeds the configured resource limit.

For a closed curve, an exactly duplicated final point equal to the first SHALL be rejected; each authoritative cyclic point is listed once. Tolerance-based point merging is forbidden.

5.3 Nonconsecutive duplicates

Nonconsecutive duplicate points SHALL be retained as distinct interpolation constraints with distinct point handles. They may imply a loop. They MUST NOT be silently deduplicated.

5.4 Shape policy

The default policy SHALL prioritize:

1. exact interpolation;
2. requested continuity;
3. regularity;
4. deterministic construction;
5. low preimage strain and curvature variation;
6. short and visually local deviations from the input polyline.

It SHALL NOT promise a simple curve. Optional policies MAY request `avoid_self_intersection`, `prefer_convex`, or `limit_turning`, but these are extra constraints that can cause a typed construction failure.

6. Public API

6.1 Constructor

`PHBSpline` is abstract. The required concrete signatures are:

```
class PHBSplineOpen(PHBSpline):
    def __init__(
        self,
        points: ArrayLike,
        *,
        g_order: int | None = None,
        c_order: int | None = None,
        curvature_order: int | None = None,
        construction: ConstructionPolicy | None = None,
        editing: EditingPolicy | None = None,
        inverse: InversePolicy | None = None,
        numerics: NumericalPolicy | None = None,
    ) -> None: ...

class PHBSplineClosed(PHBSpline):
    def __init__(
        self,
        points: ArrayLike,
        *,
        g_order: int | None = None,
        c_order: int | None = None,
        curvature_order: int | None = None,
        construction: ConstructionPolicy | None = None,
        editing: EditingPolicy | None = None,
        inverse: InversePolicy | None = None,
        numerics: NumericalPolicy | None = None,
    ) -> None: ...
```

Topology SHALL be selected only by the concrete class name; a `closed` constructor Boolean is not part of either signature. Policies SHALL be immutable dataclasses. `None` selects documented defaults.

6.2 Required properties

```
@property
def points(self) -> NDArray[np.float64]: ...       # read-only snapshot copy

@property
def point_handles(self) -> tuple[PointHandle, ...]: ...

@property
def num_points(self) -> int: ...

@property
def num_spans(self) -> int: ...                    # includes hidden spans

@property
def preimage_degree(self) -> int: ...
```

```
@property
def degree(self) -> int: ...                          # 2*m + 1

@property
def requested_continuity(self) -> ContinuitySpec: ...

@property
def verified_continuity(self) -> ContinuitySpec: ...

@property
def closed(self) -> bool: ...

@property
def length(self) -> float: ...

@property
def length_coordinate(self) -> LengthCoordinate: ...

@property
def version(self) -> int: ...

@property
def diagnostics(self) -> BuildDiagnostics: ...

@property
def last_edit_report(self) -> EditReport | None: ...
```

`points` MUST return a read-only snapshot copy and MUST NOT expose mutable storage. Its cost is `O(N)`; high-frequency consumers SHALL retain a snapshot or use point handles rather than repeatedly requesting the full array.

6.3 Scalar geometry methods

```
def point(self, u: Real) -> NDArray[np.float64]: ...
def derivative(
    self,
    u: Real,
    order: int = 1,
    *,
    side: Literal["auto", "left", "right"] = "auto",
) -> NDArray[np.float64]: ...
def tangent(self, u: Real) -> NDArray[np.float64]: ...
def normal(self, u: Real, side: Literal["left", "right"] = "left") -> NDArray[np.float64]: ...
def principal_normal(self, u: Real) -> NDArray[np.float64]: ...
def signed_curvature(self, u: Real) -> float: ...
def curvature_vector(
    self,
    u: Real,
    order: int = 0,
    *,
    side: Literal["auto", "left", "right"] = "auto",
) -> NDArray[np.float64]: ...
def curvature_derivative(
    self,
    u: Real,
    order: int = 1,
    *,
    side: Literal["auto", "left", "right"] = "auto",
```

```
) -> float: ...
def jet(
    self,
    u: Real,
    order: int,
    *,
    side: Literal["auto", "left", "right"] = "auto",
) -> tuple[NDArray[np.float64], ...]: ...
def curvature_vector_jet(
    self,
    u: Real,
    order: int,
    *,
    side: Literal["auto", "left", "right"] = "auto",
) -> tuple[NDArray[np.float64], ...]: ...
```

`derivative(u, r)` returns $D_u^r z$ for any nonnegative integer `r` permitted by `NumericalPolicy.max_evaluation_order`. Order zero is exactly `point(u)`. Differentiation is always with respect to the normalized global spline parameter `u`, never the local span coordinate. `tangent(u)` is the normalized first derivative and MUST NOT substitute for the generally nonunit `derivative(u, 1)`.

Let $C_0 = \kappa N_L = D_s^2 z$ be the ordinary curvature vector. `curvature_vector(u, j)` returns $D_u^j C_0$, not merely $(D_u^j \kappa) N_L$.

The no-keyword call `curvature_vector(u)` retains the cubic-family order-zero result. `jet(..., order=r)` returns $(z, D_u z, \dots, D_u^r z)$, and `curvature_vector_jet(..., order=j)` returns $(C_0, D_u C_0, \dots, D_u^j C_0)$. A full jet SHALL share one recurrence and workspace; it MUST NOT invoke the single-order method repeatedly.

`order` SHALL accept Python and NumPy integer scalars, reject booleans and nonintegral values, and be nonnegative. An order above the configured resource limit raises `ResourceLimitError`. A curve derivative whose order exceeds the local curve degree is the exact zero vector. Curvature and curvature-vector derivatives are rational and do not generally terminate at that degree.

At a join, `side="left"` or `side="right"` requests that one-sided value. With `side="auto"`, the evaluator SHALL return a common value only when the relevant left/right jets agree within their independently computed error bounds; otherwise it raises `DiscontinuousDerivativeError`. At an open endpoint, `auto` selects the interior side and an explicitly unavailable side is invalid. Away from a join, `side` does not change the result.

The global compatibility parameter `u` is in `[0, 1]`. It is derived from the compensated flat prefix of parameter weights. A local geometry edit can therefore change the normalized `u` assigned to downstream geometry even when that geometry is spatially unchanged. Applications requiring edit-stable references SHALL use `CurveLocation` or `PointHandle`.

For an open spline, `u=0` and `u=1` are the two endpoints. For a closed spline, `u=0` and `u=1` denote the same seam point and frame; `arc_length(0)=0` and `arc_length(1)=length`. Exact seam queries SHALL not choose inconsistent left/right branches.

6.4 Distance methods

```
def arc_length(self, u: Real, *, extended: bool = False) -> float | LengthCoordinate: ...
def parameter_at_length(self, s: Real | LengthCoordinate) -> float: ...
def location_at_length(self, s: Real | LengthCoordinate) -> CurveLocation: ...
def point_at_length(self, s: Real | LengthCoordinate) -> NDArray[np.float64]: ...
def frame_at_length(self, s: Real | LengthCoordinate) -> Frame2D: ...
def distance_between(
    self,
    u0: Real,
    u1: Real,
    *,
    mode: Literal["absolute", "signed", "forward"] = "absolute",
) -> float: ...
```

```
def advance_by_length(self, location: CurveLocation, ds: Real) -> CurveLocation: ...
def point_after_length(self, location: CurveLocation, ds: Real) -> NDArray[np.float64]: ...
```

`advance_by_length` is REQUIRED because a global binary64 distance cannot resolve every small local increment when total length is extremely large. It SHALL traverse spans sequentially from the supplied location without first forming `global_s + ds` in one float. `point_after_length(location, ds)` SHALL be equivalent to `point(advance_by_length(location, ds))`; the name deliberately avoids confusion with a geometric normal offset curve.

`distance_between` semantics are exact and SHALL be:

- `mode="signed"`: `arc_length(u1) - arc_length(u0)` on the chosen `[0, 1]` cut;
- `mode="absolute"`: the absolute value of the signed result;
- `mode="forward"`: traversal-direction distance from `u0` to `u1`, adding total length on a closed curve when the target precedes the source, and rejecting backward travel on an open curve.

6.5 Batch methods

```
def points_at(self, u: ArrayLike, *, out: NDArray[np.float64] | None = None) ->
 ↪  NDArray[np.float64]: ...
def tangents_at(self, u: ArrayLike, *, out: NDArray[np.float64] | None = None) ->
 ↪  NDArray[np.float64]: ...
def derivatives_at(
    self,
    u: ArrayLike,
    order: int = 1,
    *,
    side: Literal["auto", "left", "right"] = "auto",
    out: NDArray[np.float64] | None = None,
) -> NDArray[np.float64]: ...
def curvature_vectors_at(
    self,
    u: ArrayLike,
    order: int = 0,
    *,
    side: Literal["auto", "left", "right"] = "auto",
    out: NDArray[np.float64] | None = None,
) -> NDArray[np.float64]: ...
def points_at_length(self, s: ArrayLike, *, out: NDArray[np.float64] | None = None,
 ↪  assume_sorted: bool = False) -> NDArray[np.float64]: ...
def parameters_at_length(self, s: ArrayLike, *, out: NDArray[np.float64] | None = None,
 ↪  assume_sorted: bool = False) -> NDArray[np.float64]: ...
```

For sorted distances, the implementation SHALL use a linear span walk rather than one binary search per query.

The derivative batch methods accept one common order and return `u.shape + (2,)`. The reference profile dispatches each element through the scalar verified kernel; grouping by span and shared derivative ladders is a future throughput optimization. Scalar validation, join-side semantics, and numerical acceptance rules are identical to the corresponding scalar methods.

6.6 Editing methods

```
def point_handle(self, index: int) -> PointHandle: ...
def index_of(self, handle: PointHandle) -> int: ...

def move_point(
    self,
    point: int | PointHandle,
    value: ArrayLike,
    *,
```

```
    repair: EditRepair | None = None,
) -> EditReport: ...

def insert_point(
    self,
    index: int,
    value: ArrayLike,
    *,
    repair: EditRepair | None = None,
) -> InsertResult: ...

def delete_point(
    self,
    point: int | PointHandle,
    *,
    repair: EditRepair | None = None,
) -> EditReport: ...

def append_point(self, value: ArrayLike, *, repair: EditRepair | None = None) -> InsertResult:
 ↪  ...
def prepend_point(self, value: ArrayLike, *, repair: EditRepair | None = None) -> InsertResult:
 ↪  ...

def edit(self, *, repair: EditRepair | None = None) -> EditTransaction: ...
```

Insertion index semantics SHALL match `list.insert`: insert before the current item at `index`; `index == num_points` appends.

Every mutation SHALL be atomic. On any exception, input points, point handles, geometry, metric prefixes, version, and caches SHALL remain exactly as before the operation.

6.7 Snapshots

```
def snapshot(self) -> PHBSplineSnapshot: ...
```

A snapshot SHALL be immutable with respect to later source-object edits and SHALL expose the scalar and batch query API and `offset(distance)` but no mutation API. The reference snapshot retains the already-published immutable spans and arrays.

6.8 Stable handles and locations

```
@dataclass(frozen=True, slots=True)
class PointHandle:
    id: int

@dataclass(frozen=True, slots=True)
class CurveLocation:
    span_id: int
    local_u: float
    version: int

@dataclass(frozen=True, slots=True)
class LengthCoordinate:
    hi: float
    lo: float
```

A deleted `PointHandle` SHALL raise `StaleHandleError`. A `CurveLocation` from an older version SHALL raise `StaleLocationError` unless the implementation can prove that its span identity and local geometry were preserved.

6.9 Exact offset and read-only NURBS handle

Both mutable splines and immutable snapshots SHALL expose:

```
def offset(self, distance: Real) -> NURBSHandle: ...
```

The receiver is the input spline handle. `distance` is a finite signed length in user coordinates. Positive values select the package left normal and negative values select the right normal. The returned handle captures one complete committed source version atomically and has no live mutable back-reference.

The cubic and PH B-spline families SHALL return the same public type:

```
class NURBSHandle:
    @property
    def degree(self) -> int: ...

    @property
    def knots(self) -> NDArray[np.float64]: ...

    @property
    def control_points(self) -> NDArray[np.float64]: ...

    @property
    def weights(self) -> NDArray[np.float64]: ...

    @property
    def num_control_points(self) -> int: ...

    @property
    def num_spans(self) -> int: ...

    @property
    def domain(self) -> tuple[float, float]: ...

    @property
    def closed(self) -> bool: ...

    def point(self, u: Real) -> NDArray[np.float64]: ...
```

This is deliberately a small read-only interface. It SHALL NOT expose edits, fitting, derivatives, arc-length operations, control-point setters, or source mutation. `domain` is exactly `(0.0, 1.0)`. The arrays have these structural contracts:

```
knots.shape == (num_control_points + degree + 1,)
control_points.shape == (num_control_points, 2)
weights.shape == (num_control_points,)
```

They contain finite binary64 values and are returned as read-only snapshots. The knot vector is nondecreasing and every weight is strictly positive. `point(u)` returns a new or read-only binary64 array of shape `(2,)` and uses the same scalar argument rules as the source `point(u)`. Section 15.6 defines the complete construction, evaluation, and failure contract.

7. Policy dataclasses

The package SHALL expose policies at public or advanced-public scope. Defaults below are normative starting values and MAY be tuned only with benchmark and test evidence.

7.1 `ConstructionPolicy`

```
@dataclass(frozen=True, slots=True)
class ConstructionPolicy:
```

```
    parameterization: Literal["centripetal", "chord", "uniform"] = "centripetal"
    max_iterations: int = 48
    max_line_search_steps: int = 16
```

All three fields affect construction. Random initialization is forbidden; the reference profile always uses the deterministic guide projection in Section 10.7. A new shape objective or adaptive-refinement control SHALL become public only together with its implementation, specification, and regression tests.

7.2 EditingPolicy

```
@dataclass(frozen=True, slots=True)
class EditingPolicy:
    default_repair: Literal["strict_local", "expand", "global"] = "strict_local"
    initial_patch_spans: int | None = None        # None -> 2*(m + 3)
    max_patch_spans: int = 64
```

`strict_local` SHALL never silently perform a global rebuild. In the reference profile, the geometric patch is always the order-derived patch in Section 11.3. `initial_patch_spans` and `max_patch_spans` are post-construction admission limits, not patch-size selectors; `expand` admits that same patch against `max_patch_spans`. `global` rebuilds the entire spline.

7.3 InversePolicy

```
@dataclass(frozen=True, slots=True)
class InversePolicy:
    lut_nodes_min: int = 8
    lut_nodes_max: int = 128
    lut_power_of_two: bool = True
    fast_iterations: int = 2
    max_iterations: int = 67
    endpoint_reverse_threshold: float = 0.5
```

Every field affects the reference inverse. It uses a linear length-within-LUT-cell seed, Newton correction, and bisection fallback.

7.4 NumericalPolicy

```
@dataclass(frozen=True, slots=True)
class NumericalPolicy:
    regularity_ratio_min: float = 1.0e-12
    max_preimage_degree: int = 16
    max_evaluation_order: int = 64
    max_regularization_subdivision_depth: int = 24
    parameter_ulp_slack: int = 4
    position_eps_factor: float = 256.0
    continuity_eps_factor: float = 1024.0
    reject_unresolved_global_lengths: bool = True
```

Requests requiring a preimage degree above `max_preimage_degree` SHALL raise `ResourceLimitError` before construction begins. Users MAY explicitly provide a higher limit.

`max_evaluation_order` is a resource guard, not a mathematical restriction on the API. Users MAY explicitly increase it. Implementations SHALL reject an order before allocating order-dependent work when it exceeds the configured limit.

Every public numerical-policy field affects the reference implementation.

8. Internal architecture

8.1 Layer separation

The package SHALL separate four layers:

1. **Interpolation model:** user points, point handles, continuity request, policies, and optional Hermite constraints.
2. **Authoritative PH spline:** knot topology and complex preimage coefficients or equivalent compatible span data.
3. **Compiled span kernels:** position, preimage, speed, arc length, inverse LUT, and regularity bounds.
4. **Published indexes:** read-only parameter and compensated arc-length prefix arrays.

A query SHALL never invoke the nonlinear construction solver.

8.2 Reference module layout

```
ph_spline/
    __init__.py
    base.py                 # sibling PHSpline abstract base
    bspline.py              # PHBSpline family API, topologies, edits and queries
    bspline_types.py        # policies, handles, reports, diagnostics
    bspline_basis.py        # basis, twisted extraction, exact constraints, solve
    bspline_construction.py # validation, guide, build, local repair, verification
    bspline_segment.py      # immutable compiled PH span and inverse kernel
    exceptions.py
    arclength.py            # compensated prefix helper shared with cubic
    nurbs.py                # shared immutable NURBS handle and exact PH offsets
    _constants.py
    py.typed
```

Circular imports SHALL be avoided. `bspline_basis.py`, `bspline_construction.py`, and `bspline_segment.py` SHALL not import the mutable API layer.

8.3 Reference flat indexes and future dynamic tree

The reference profile SHALL publish contiguous arrays for user knots, midpoint-refined span knots, normalized length prefixes, and physical length prefixes. Scalar parameter and distance lookup use binary search and are $O(\log M)$. Prefixes SHALL be accumulated by the shared compensated-sum helper and SHALL be strictly increasing in both normalized and user coordinates when `reject_unresolved_global_lengths=True`.

Local edits structurally share unchanged immutable span kernels, but the reference implementation validates the complete point array and rebuilds the flat parameter and length arrays before atomic publication. This work is $O(N)$. Documentation and benchmarks MUST distinguish local nonlinear/span work from total edit latency.

A balanced augmented tree or rope that reduces publication to $O(\log N)$ is a **future extension**. Such a replacement must preserve exact lookup boundary semantics, stable handles, snapshot behavior, compensated length totals, and all verification gates before it may claim conformance. The earlier statement that a flat prefix array was forbidden was aspirational and did not describe the tested implementation.

Stable handles are stored in sequence order in the reference object. Lookup by handle is linear; insertion and deletion preserve every surviving handle's identity. A future locator table/tree MAY improve this complexity without changing semantics.

8.4 Compiled span kernel

Each immutable `SpanKernel` SHALL contain at least:

```
span_id
left_point_id / right_point_id or hidden-span metadata
parameter_width h
```

```
preimage degree m
preimage Bernstein coefficients b[0:m+1]
canonical left/right preimage jets through order m-1
position Bernstein coefficients p[0:2m+2]
speed Bernstein coefficients r[0:2m+1]
arc Bernstein coefficients a[0:2m+2]
forward and reverse Bernstein arc coefficients
optional power coefficients for low-degree fast seeds
span length as float64
regularity lower and upper bounds
inverse LUT
```

The arrays SHALL be read-only after construction.

9. Coordinate normalization and scaled representation

9.1 Reference global normalization

The reference constructor uses one affine frame:

$$O = P_0, \qquad H = \max_i \|P_{i+1} - P_i\|_2, \qquad \widehat{P}_i = (P_i - O)/H.$$

Safe component differences and `hypot`-style chord norms SHALL be used before normalization. H must be positive and finite. Construction, compilation, regularity certification, and interpolation verification operate in this normalized frame; physical positions and lengths are restored with O and H.

9.2 Edit-frame stability

Local edits reuse the already-published (O, H) frame so unchanged exterior span arrays remain bitwise identical. The reference profile does not perform an automatic whole-curve rebase. An edit whose new coordinates cannot be represented safely in the retained frame SHALL fail atomically rather than overflow or silently rebuild exterior geometry.

Per-patch power-of-two frames are a future extension. They require explicit cross-frame boundary-jet conversion and cannot be retrofitted by changing the normalization silently.

Physical length and curvature scale as

$$L = H\widehat{L}, \qquad \kappa = \hat{\kappa}/H.$$

9.3 Endpoint anchoring

Every span SHALL store exact references to its structural endpoints. `point(0)`, `point(1)`, and exact interpolation-knot queries SHALL return the stored input point values.

This endpoint return is permitted only after the independently evaluated polynomial displacement residual satisfies the interpolation tolerance. Endpoint snapping MUST NOT conceal a failed solve.

10. Initial construction

10.1 Construction pipeline

The constructor SHALL execute the following stages:

1. validate and canonicalize input;
2. determine continuity request and degree;
3. normalize points and construct guarded parameter widths;
4. construct guide velocity samples and select the square-root seam sign;

5. build the minimum-degree open or twisted-closed simple-knot basis, including the midpoint of every user interval;
6. seed its controls by sign-aware interpolation at unwrapped Greville abscissae;
7. project the seed onto all exact PH displacement constraints;
8. Bezier-extract every span directly from endpoint basis derivatives;
9. canonicalize shared endpoint preimage jets and compile immutable span kernels;
10. independently verify interpolation, signed seam continuity, regularity, and metric postconditions;
11. build flat parameter and compensated length prefixes;
12. assign handles and publish only the complete verified state.

The midpoint basis is therefore built **before** the nonlinear solve. Solving on one topology and inserting midpoint knots afterwards is not the reference algorithm and changes both the unknown count and the constraint Jacobian.

No partially initialized public object may escape.

10.2 Parameter weights

For chord lengths $d_i > 0$, initial span weights SHALL be:

- uniform: $h_i = 1$;
- chord: $h_i \propto d_i$;
- centripetal: $h_i \propto \sqrt{d_i}$.

Weights SHALL be positive and finite. For required order $m > 1$, the reference profile limits the adjacent/global parameter-width dynamic range before forming high-order physical jets:

$$R_m = (10^8)^{1/(m-1)}, \qquad h_i \leftarrow \max\left(h_i, \frac{\max_j h_j}{R_m}\right).$$

This changes only parameter allocation, not input geometry or exact interpolation. It bounds factors of the form $h^{-(m-1)}$ that would otherwise make high-order seam and patch jets numerically unrepresentable. Prefixes are formed by compensated summation and the normalized public knots must remain strictly increasing.

10.3 Guide construction

The guide exists only to choose a square-root branch and initialize the projection. It is not authoritative geometry. Define interval secants

$$d_i = \frac{P_{i+1} - P_i}{h_i}.$$

At an interior point, including every point of a closed curve, use

$$v_i = \frac{h_i d_{i-1} + h_{i-1} d_i}{h_{i-1} + h_i}.$$

For an open curve use $v_0 = d_0$ and $v_N = d_{N-1}$. If the weighted average has norm no greater than $32\epsilon \max(|d_{i-1}|, |d_i|)$, replace it by the larger adjacent secant, with the later secant winning only when it is strictly larger. This deterministic reversal fallback avoids an artificial zero guide speed.

10.4 Square-root branch initializer

For each guide velocity v_i, compute one principal complex square root $r_i = \sqrt{v_i}$. For an open curve choose signs greedily so consecutive roots have minimum Euclidean distance.

For a closed curve, greediness is insufficient because the end condition is cyclic. The implementation SHALL solve both monodromy cases $\eta \in \{+1, -1\}$ by a deterministic two-state dynamic program. With $s_i \in \{+1, -1\}$, minimize

$$\sum_{i=1}^{N-1} |s_i r_i - s_{i-1} r_{i-1}|^2 + |s_{N-1} r_{N-1} - \eta r_0|^2.$$

Choose the lower-cost pair `(sign sequence, eta)`; exact ties SHALL prefer $\eta = +1$. The chosen η is stored as authoritative topology and used by basis extraction, Greville seeding, seam canonicalization, verification, and wrapped local edits. It is not a disposable initializer detail.

The guide roots are linearly interpolated in the complex plane at the basis Greville abscissae. For a closed basis the Greville values SHALL remain unwrapped. If $x = qT + \bar{x}$, evaluate the seed as

$$c^{(0)}(x) = \eta^q \operatorname{lerp}(r(\bar{x})).$$

Reducing the abscissa modulo T without the factor η^q is incorrect for an antiperiodic basis.

10.5 Exact interpolation constraints

For each input interval with displacement

$$D_i = P_{i+1} - P_i,$$

the constraint is

$$F_i(c) = \int_{\tau_i}^{\tau_{i+1}} w(t)^2 \, dt - D_i = 0.$$

In local span coordinates:

$$F_i(c) = \sum_{j \in \mathcal{S}_i} \int_0^1 h_j w_j(\nu)^2 \, d\nu - D_i,$$

where $\mathcal{S}_i$ contains the visible and hidden spans between the two user anchors.

These are two real equations per input interval. Their support is local in the preimage control ordering.

In the reference topology, $\mathcal{S}_i$ contains exactly the two midpoint subspans. If $b_0, \ldots, b_m$ are the extracted Bernstein controls on one subspan, the integral is evaluated with the analytic Bernstein Gram matrix

$$G_{ab} = \int_0^1 B_a^m(\nu) B_b^m(\nu) d\nu = \frac{\binom{m}{a}\binom{m}{b}}{(2m+1)\binom{2m}{a+b}},$$

so its displacement contribution is $h\, b^T G b$. No quadrature or sampled constraint is permitted.

10.6 Analytic Jacobian

For a local Bezier preimage coefficient $b_k = x_k + i y_k$:

$$\frac{\partial F}{\partial x_k} = 2h \int_0^1 w(\nu) B_k^m(\nu) \, d\nu,$$

$$\frac{\partial F}{\partial y_k} = 2ih \int_0^1 w(\nu) B_k^m(\nu) \, d\nu.$$

For global B-spline controls, multiply by the Bezier extraction matrix. All integrals SHALL be evaluated exactly through Bernstein products and coefficient sums.

Finite-difference Jacobians are forbidden in the production solver. Complex-step differentiation MAY be retained as an independent debug/test oracle, as in the baseline package.

10.7 Reference shape selection

The reference profile does not run a separate elastica, strain, length, or curvature optimizer. Shape is selected by starting at the deterministic guide controls and repeatedly applying the minimum-Euclidean-norm linearized constraint correction. Equivalently, each full-construction step solves

$$\min_{\delta c} \|\delta c\|_2 \quad \text{subject to} \quad J_F \delta c = -F.$$

This local projection keeps the converged branch near the guide without introducing additional objective weights. No shape-objective argument is exposed. Adding a fairness objective changes the specified curve and requires a documented implementation and new regression fixtures before it becomes part of the public API.

10.8 Nonlinear method

For a full open or closed construction, form the real constraint Jacobian J_F and solve on the smaller constraint space:

$$J_F J_F^T \lambda = -F, \qquad \delta c = J_F^T \lambda.$$

Try deterministic relative diagonal damping values $0, 10^{-15}, 10^{-13}, 10^{-11}, 10^{-9}$, scaled by $\max(1, \|J_F J_F^T\|_{\max})$, until a finite sparse solve is obtained. Apply bounded backtracking factors $1, 1/2, 1/4, \ldots$ and accept only a finite trial whose real residual 2-norm is strictly smaller.

For a local edit patch, impose each exterior endpoint preimage jet through order $m-1$ by directly solving the clamped endpoint extraction block and eliminate those controls from the unknown vector. Solve the remaining small system $J_{F,\text{free}} \delta c = -F$ with rank-revealing dense least squares. This deliberately avoids forming normal equations for high-order endpoint-constrained patches, which would square their condition number.

All paths SHALL use analytic residuals/Jacobians, finite checks, hard iteration and line-search bounds, and independent post-verification. A solver success flag alone is never acceptance.

For fixed degree, bounded refinement, and bounded iterations, initial construction is `O(N)` in point count.

10.9 Minimum-degree hidden-knot allocation

The base topology SHALL contain one simple midpoint knot in every input interval. This fixed allocation supplies the degrees of freedom needed for exact displacement interpolation while retaining the minimum preimage degree $m = r_*$. All extraction SHALL be performed directly from endpoint basis jets; an ill-conditioned Bernstein collocation inverse is not acceptable for high-order requests.

For each nonzero span, evaluate all active B-spline basis derivatives through order m at the left endpoint using the standard triangular basis-derivative recurrence. Convert that Taylor jet to Bernstein coefficients with the exact finite basis transformation. For a closed basis, apply the twisted extension $c_{j+qn} = \eta^q c_j$ to every wrapped extraction column before combining equal base indices.

After extraction, the two independently rounded representations of a shared knot jet SHALL be replaced by one canonical physical value. Map the right value into the left gauge using the join sign, average after magnitude scaling to avoid overflow/cancellation, and store the same physical jet back on both sides with their local-width factors. At the closed seam the join sign is η; at every other join it is $+1$.

If this topology cannot satisfy the interpolation residual or regularity certificate, construction SHALL raise the corresponding typed error. It MUST NOT increase m, reduce continuity, or accept a near-cusp. A future adaptive simple-knot refinement policy MAY add further local knots without changing m, provided the actual hidden-span count is reported.

Counterexample (collocation extraction). Solving for a Bernstein extraction matrix from sampled basis values can be adequate at low order but becomes ill-conditioned at G8: a few input ulps are amplified into visible join residuals. Direct endpoint-derivative extraction plus canonical shared jets removes this avoidable numerical disagreement.

11. Local editing

11.1 Transaction model

Every edit SHALL construct private candidate points, span tuple, prefixes, and diagnostics. Commit publishes the complete candidate and increments the version only after verification. On failure, points, handles, span objects, prefixes, version, and caches remain unchanged.

A transaction may batch multiple point changes:

```
with curve.edit(repair="strict_local") as tx:
    tx.move_point(handle_a, [x1, y1])
    tx.insert_point(index_b, [x2, y2])
    tx.delete_point(handle_c)
```

The union of affected neighborhoods is solved once by the reference transaction.

11.2 Locality invariant

Let a patch cover the parameter interval $[a, b]$. A committed local edit SHALL satisfy:

- all preimage coefficients and span kernels outside the patch are unchanged;
- required preimage or curve jets at the patch boundaries match the frozen exterior data;
- the patch boundary positions are fixed unless an endpoint of the whole open spline is intentionally moved;
- the total patch displacement equals the new boundary displacement.

For a purely interior point move, exterior boundary points are unchanged, so

$$\int_a^b (w_{\text{new}}(t)^2 - w_{\text{old}}(t)^2)\, dt = 0.$$

This closure condition prevents a local hodograph change from translating the entire downstream curve.

Unchanged span objects SHALL be structurally shared and therefore bitwise identical.

11.3 Patch size

The initial patch SHALL include:

- every input interval whose displacement constraint changed;
- every span whose basis support intersects a variable preimage control;
- any hidden spans belonging to those input intervals.

With one midpoint knot per input interval, an interior patch with both exterior jets fixed needs at least m logical intervals for constraint closure. This count follows from the complex unknowns: a clamped patch of K logical intervals has $2K + m$ controls; fixing m controls at each end leaves $2K - m$ controls for K complex displacement constraints, so $2K - m \geq K$ requires $K \geq m$. The reference default uses three additional guard intervals, $K = m + 3$, for guide shape freedom and regularity margin, hence $2(m + 3)$ compiled spans. It therefore rebuilds 10, 14 and 22 spans for G2, G4 and G8 respectively. The count depends on requested order; a fixed count independent of m is not a valid general rule.

For open curves, grow the shortest contiguous interval containing every changed displacement until it reaches K, preferring the left side and then the right side on each growth pass where available. For closed curves, cut the cycle through the largest unchanged gap, then grow the resulting cyclic patch alternately left and right. A local closed patch must leave at least one certified exterior interval; otherwise strict-local reconstruction fails.

11.4 Closed wrapped-patch gauge

A closed local patch is solved as an ordinary clamped open preimage in a continuous lifted gauge. If the patch crosses the stored seam, its right boundary physical jets SHALL be multiplied by every crossed monodromy factor η. After the patch solve, extracted preimages on the portion lying after the seam are multiplied by η to return them to the global stored gauge. This sign change leaves w^2, $|w|^2$, positions, speed, and arc length unchanged.

Verification SHALL cover every join adjacent to a rebuilt span, including both patch/exterior boundaries and the closed seam, using sign η only at the global seam. Checking the clamped patch internally while omitting its wrapped exterior joins is insufficient.

11.5 Move operation

`move_point` SHALL:

1. resolve the stable handle or index;
2. validate the new point against adjacent points;
3. update only the two adjacent displacement constraints for an interior open point, one for an open endpoint, or two cyclic constraints for a closed curve;
4. construct a candidate local guide and preimage seed;
5. freeze exterior controls/jets;
6. solve and verify the patch;
7. recompile affected spans and inverse kernels;
8. assemble and validate new flat parameter/length prefixes;
9. commit and increment `version`.

11.6 Insert operation

`insert_point(index, value)` SHALL:

1. identify the old input interval being split;
2. assign a new stable point ID;
3. recompute the affected parameter widths from the configured rule;
4. replace one displacement constraint by two;
5. construct the order-derived clamped patch and guide seed;
6. solve and verify the patch;
7. assemble prefixes and commit atomically.

Insertion MUST NOT renumber existing point handles.

11.7 Delete operation

`delete_point` SHALL:

1. reject deletion if it would leave too few points;
2. merge the two adjacent displacement constraints into one;
3. recompute affected parameter widths and construct the clamped patch guide;
4. solve and verify the bounded patch;
5. assemble prefixes;
6. invalidate the deleted handle only after commit.

11.8 Edit complexity contract

For fixed continuity order and the reference patch $K = 2(m + 3)$ compiled spans, the nonlinear solve and span compilation cost is bounded independently of total point count. The current total edit cost is nevertheless

$$T_{\text{edit}} = O(N) + O(Km^3 I),$$

because the reference implementation validates/normalizes the whole point sequence, assembles the full span tuple, and rebuilds flat parameter and length prefixes. A future tree profile may reduce the publication term to $O(\log N)$, but that complexity MUST NOT be attributed to this profile.

This bound is obtained by allowing failure. If the strict patch has no verified solution, `LocalEditFailure` SHALL be raised and the object SHALL be unchanged.

In this profile, `expand` changes the patch admission limit but does not retry with progressively larger geometry; `global` performs an explicit full solve. True geometric expansion is a future extension.

No documentation may claim unconditional `O(1)` or `O(log N)` successful editing for arbitrary point displacement without stating this policy distinction.

12. Regularity certification

12.1 Required property

Every committed span SHALL satisfy

$$|w(t)| > 0$$

throughout the closed span, with a quantitative margin. Sampling alone is insufficient.

12.2 Recursive Bernstein-box certificate

A Bezier polynomial lies in the convex hull of its control points and hence inside their axis-aligned complex-plane bounding box. The reference certificate deliberately uses the cheaper box distance, which is a conservative lower bound on convex-hull distance. For each preimage span:

1. compute the minimum axis-aligned box containing its complex Bernstein controls;
2. compute the Euclidean distance d from the origin to that box;
3. if $d > 0$, then $|w(\nu)| \geq d$ on that subspan;
4. otherwise subdivide the preimage by de Casteljau at $\nu = 1/2$ and repeat;
5. stop when all subspans are certified or the subdivision-depth limit is reached.

If the origin remains inside or numerically indistinguishable from the hull at the depth limit, construction SHALL fail. The exception is `NonRegularSplineError`.

12.3 Relative regularity margin

Let $d_{\min}$ be the certified lower bound and $d_{\max}$ an upper bound from the maximum control radius after subdivision. Define

$$\rho = \frac{d_{\min}^2}{d_{\max}^2}.$$

The span SHALL satisfy

$$\rho \geq \rho_{\min},$$

with default $\rho_{\min} = 10^{-12}$. This protects distance inversion and curvature evaluation from near-cuspidal conditioning.

Users MAY request a stricter margin. Weakening the default requires an explicit advanced policy and SHALL be reflected in diagnostics.

12.4 Speed evaluation

Runtime speed SHALL be evaluated from the preimage,

$$\sigma = h|w|^2,$$

not solely from a potentially cancellation-prone expanded speed polynomial. Use scaled complex evaluation and `hypot`-style normalization before squaring.

13. Exact forward arc-length compilation

13.1 Authoritative metric

For every span, the speed and arc length SHALL be constructed by exact polynomial coefficient operations from the preimage. No adaptive quadrature, tessellation, or chord-length approximation may contribute to the authoritative length.

13.2 Forward and reverse arc polynomials

Each span SHALL compile:

$$S_f(\nu) = \int_0^{\nu} h|w(v)|^2 dv,$$

and

$$S_r(v) = \int_0^{v} h|w(1-q)|^2 dq = L - S_f(1-v).$$

`S_r` SHALL be stored as the Bernstein sequence $L - S_f(1-v)$, formed once at span compilation. Near the right endpoint, inversion and residual checks SHALL use S_r and target $L - s$, never a runtime subtraction of two nearly equal forward lengths.

13.3 Polynomial evaluation strategy

Each span SHALL store forward/reverse Bernstein arc coefficients. The authoritative forward and reverse evaluations use de Casteljau. For curve degree at most 17 the compiler also stores power coefficients used only by the unverified fast inverse seed; if a fast speed result is nonpositive or nonfinite, the kernel immediately evaluates $h|w|^2$ instead.

13.4 Length aggregation

Span lengths SHALL be aggregated into a flat binary64 prefix with Neumaier compensation. Both normalized and user-space prefixes must be finite and strictly increasing. The reference `LengthCoordinate` interface currently returns `(hi=value, lo=0)` and is API scaffolding, not a claim of implemented double-double storage. Double-double tree aggregation is a future extension.

14. Random distance access

14.1 Global algorithm

`location_at_length(s)` SHALL perform:

1. validate and canonicalize `s` as float or `LengthCoordinate`;
2. divide by the global spatial scale and binary-search the normalized flat prefix for the containing span in $O(\log M)$;

3. subtract the normalized prefix to obtain the local target;
4. binary-search the span's monotone LUT length samples;
5. linearly interpolate a parameter seed inside that LUT bracket;
6. apply bracketed Newton correction;
7. use bisection whenever the Newton proposal is nonfinite or leaves the bracket;
8. verify the forward or reverse arc residual;
9. return `CurveLocation(span_id, local_u, version)`.

`point_at_length` then evaluates one span kernel.

14.2 Per-span inverse LUT

At construction, choose a power-of-two node count between policy limits from the certified preimage variation estimate

$$V = \max\left(1, \frac{d_{\max}}{\max(d_{\min}, 10^{-300})}\right), \qquad M_0 = \max\left(M_{\min}, \left\lceil 4 + 2\sqrt{V} \right\rceil\right).$$

If power-of-two LUTs are requested, round M_0 upward to the next power of two, then set $M = \min(M_{\max}, M_0)$. Thus the resource cap is authoritative even when the variation heuristic requests more nodes.

Store uniformly spaced parameter nodes $\nu_j = j/M$ and the directly evaluated monotone arc values $S_f(\nu_j)$. Force the two endpoint entries to exactly zero and L, and reject a non-strictly-increasing table. Runtime cell selection is a binary search in stored arc values. The LUT supplies a bracket and seed only; authoritative acceptance uses Bernstein arc evaluation.

14.3 Linear bracket seed

Linearly interpolate ν between the two LUT arc values bracketing the target. The result lies inside the bracket by construction. For curve degree at most 17, the reference kernel MAY apply `fast_iterations` unverified Horner Newton steps as a seed improvement, but those steps SHALL NOT alter the LUT bracket or satisfy the final acceptance gate.

The LUT is an accelerator, not an authority. An inaccurate seed cannot compromise correctness because the subsequent solver remains bracketed.

14.4 Safeguarded correction

Let

$$f(\nu) = S(\nu) - s_{\text{local}}, \qquad f'(\nu) = h|w(\nu)|^2,$$

with the corresponding reverse definitions when solving from the right.

The reference Newton proposal is

$$\nu_N = \nu - \frac{f}{f'}.$$

It SHALL be replaced by the bracket midpoint if nonfinite or outside the current bracket.

The bracket SHALL be updated after every evaluation. Iteration SHALL have a hard maximum. Failure to satisfy the residual bound raises `ArcLengthInversionError`; an unverified parameter is never returned.

14.5 Endpoint reversal

If $s_{\text{local}} > L/2$, use

$$v = 1 - \nu, \qquad S_r(v) = L - s_{\text{local}}.$$

Recover $\nu = 1 - v$ only after the reversed solve. Exact endpoints SHALL return `0.0` or `1.0` without iteration.

14.6 Residual tolerance

Let ϵ be binary64 machine epsilon. Accept only if

$$|S(\nu) - s_{\text{local}}| \le 64\epsilon L + 4\,\text{ulp}(s_{\text{local}}).$$

For reversed evaluation, use the corresponding reversed target and error bound.

For analysis, the corresponding parameter error is bounded approximately by

$$|\delta\nu| \lesssim \frac{\text{arc residual}}{\min f'}.$$

14.7 Global length resolution

A binary64 scalar cannot distinguish two global distances separated by less than one ULP of the accumulated prefix. The reference implementation detects this at construction/edit publication by requiring every physical prefix increment to be strictly positive when `reject_unresolved_global_lengths=True`.

If the condition fails, construction/edit SHALL raise `LengthResolutionError`; it does not publish a curve with ambiguous scalar prefixes. Relative `advance_by_length` traversal remains useful because it works from one local span and does not first form a large global sum.

A future double-double index may instead retain the curve and recommend:

- `LengthCoordinate` input;
- `advance_by_length` from a local location;
- a compact local subcurve.

It SHALL NOT silently select an arbitrary neighboring span.

15. Geometry evaluation

15.1 Position

Position SHALL be evaluated with de Casteljau or a numerically equivalent stable Bezier method in the local span frame. Exact structural endpoints return stored anchors after verification.

15.2 Arbitrary-order derivative vectors

Let a compiled span have curve degree $n = 2m+1$ and local position Bernstein coefficients $p_0, \dots, p_n$. For $0 \le r \le n$, the elementary Bezier derivative formula is

$$D_\nu^r z(\nu) = \frac{n!}{(n-r)!} \sum_{a=0}^{n-r} \Delta^r p_a B_a^{n-r}(\nu),$$

and $D_\nu^r z = 0$ for $r > n$. For $r \ge 1$, the PH representation gives the algebraically equivalent identity

$$D_\nu^r z(\nu) = h \sum_{j=0}^{r-1} \binom{r-1}{j} D_\nu^j w(\nu)\, D_\nu^{r-1-j} w(\nu).$$

The reference runtime evaluates $D_\nu^j w$ from finite Bernstein-difference ladders and uses the PH Leibniz identity for every parameter derivative of positive order. Endpoint requests use the canonical stored preimage jets when available. This avoids cancellation in high-order position-control differences. Finite differences and generic polynomial root/differentiation routines are forbidden.

For the global B-spline parameter t, the local affine map gives

$$D_t^r z = h^{-r} D_\nu^r z.$$

If $H_t = \sum_i h_i$ is the unnormalized parameter total and H_x the spatial normalization scale, the public normalized-parameter derivative for $r \geq 1$ is evaluated as

$$D_u^r z = H_x H_t \left(\frac{H_t}{h_i}\right)^{r-1} \sum_{j=0}^{r-1} \binom{r-1}{j} D_\nu^j w \, D_\nu^{r-1-j} w.$$

The width-ratio clamp in Section 10.2 bounds the dominant high-order scale growth. Every final scalar/vector is checked for finiteness and raises `NumericalPrecisionError` if it is not representable.

15.3 Tangent

Evaluate the preimage $w = a + ib$, scale it by its norm, and square the normalized complex number:

$$T = \left(\frac{w}{|w|}\right)^2.$$

To reduce cancellation, compute

$$T_x = (r-s)(r+s), \qquad T_y = 2rs,$$

where $r = a/|w|$, $s = b/|w|$. Renormalize only when the squared norm differs from one by more than a documented rounding threshold. This optimized kernel SHALL agree, within the combined error bounds, with

$$\frac{\text{derivative}(u,1)}{\|\text{derivative}(u,1)\|}.$$

15.4 Curvature and curvature-vector derivatives

With derivatives taken with respect to the global B-spline parameter t, define

$$A(t) = \text{Im}(\overline{w(t)} w'(t)), \qquad B(t) = |w(t)|^2.$$

The signed curvature is the elementary rational expression

$$\kappa(t) = 2A(t)B(t)^{-2}.$$

In complex-vector form, multiplication by i rotates left by 90 degrees, so the order-zero curvature vector has the direct expression

$$C_0(t) = \kappa N_L = 2A(t)\, i\, w(t)^2 B(t)^{-3} = D_s^2 z(t).$$

The requested higher-order curvature vector is differentiated with respect to the normalized global spline parameter:

$$C_j = D_u^j C_0.$$

Construct truncated series for A, B, w^2, and B^{-1}, form $C_0 = D_s^2 z$, and then obtain its parameter derivatives by finite series operations with the required local/global scale conversion. `curvature_derivative` uses the same $2AB^{-2}$ series but returns the parameter derivative of the scalar curvature; a curvature-vector derivative is not formed by multiplying that scalar by a fixed normal.

Preimage derivatives SHALL be evaluated from Bernstein forward differences. Taylor reciprocal SHALL begin only after span regularity has established a positive speed. Normalized Taylor coefficients (derivative divided by factorial) are used in the finite series recurrences. All returned values are checked for finite representability. A structurally zero result remains exact zero; a nonstructural near-zero result retains its sign and direction and MUST NOT be snapped to zero.

The scalar curvature and order-zero vector kernels MAY use shorter specialized formulas, but they SHALL share coefficient conventions and error bounds with the arbitrary-order kernel. Curvature at an inflection is exactly or numerically zero; `principal_normal` is undefined when curvature is zero within the curvature-zero bound and SHALL raise `UndefinedPrincipalNormalError`. The curvature vector itself remains defined and equals zero at an exact inflection.

15.5 Normals

The left normal is

$$N_L = (-T_y, T_x),$$

and the right normal is $-N_L$. The principal normal is

$$N_P = \operatorname{sign}(\kappa) N_L.$$

15.6 Offsets

Exact rational NURBS offsets are REQUIRED. They are a defining production advantage of PH splines over ordinary polynomial splines, whose offsets do not generally have exact rational parameterizations.

15.6.1 Signed geometric contract

For a finite signed distance d in user coordinates, the required curve is

$$\boxed{z_d(u) = z(u) + dN_L(u), \qquad 0 \le u \le 1.}$$

Positive d uses the package left normal $N_L = (-T_y, T_x)$; negative d uses the right normal. Albrecht et al., Section 4.2, use $(y', -x')/\|z'\|$, which is this package's right normal. Code copied from that reference SHALL use its signed distance $h = -d$.

`distance` follows the scalar rules in Section 2.1: accept Python and NumPy real scalars, reject Booleans, arrays, sequences, NaN, and infinities. Zero is valid and follows the same deterministic construction. It MUST NOT select an undocumented reduced-degree representation.

For $d \neq 0$, the source SHALL have one common traversal unit tangent at every internal join and at a closed seam. If an explicitly low-continuity source has different one-sided tangents, its pointwise normal and connected parallel curve are not unique there; raise `DiscontinuousDerivativeError`. The case $d = 0$ remains defined for every regular position-continuous source.

15.6.2 Exact rational identity and degree

On a compiled span, use local coordinate ν and define

$$v(\nu) = D_\nu z(\nu), \qquad \sigma(\nu) = \|v(\nu)\| = h|w(\nu)|^2 > 0,$$

with the left rotation $R_L(x, y) = (-y, x)$. Then

$$\boxed{z_d(\nu) = \frac{\sigma(\nu) z(\nu) + d R_L(v(\nu))}{\sigma(\nu)}.}$$

If the preimage degree is m, the source curve degree is $p = 2m + 1$, both v and σ have degree $p - 1 = 2m$, and the unreduced rational offset degree is

$$\boxed{q = 2p - 1 = 4m + 1.}$$

The denominator is the positive source speed and does not depend on d. Sampling, normal-vector interpolation, least-squares fitting, and any generic approximate offset algorithm are forbidden.

15.6.3 Normative homogeneous Bernstein construction

Let one compiled source span have degree-p position controls $C_0, \dots, C_p$, degree-$(p-1)$ derivative controls

$$V_i = p(C_{i+1} - C_i), \qquad i = 0, \dots, p-1,$$

and degree-$(p-1)$ speed controls $s_0, \dots, s_{p-1}$. The s_i SHALL come from the stored PH speed product $h|w|^2$. The V_i SHALL be independently checked against the stored hodograph hw^2 before offset publication.

For each $k = 0, \dots, q$, and each valid pair $i + j = k$, define

$$\lambda_{i,j}^{(k)} = \frac{\binom{p-1}{i}\binom{p}{j}}{\binom{q}{k}}.$$

The degree-q homogeneous NURBS controls $O_k = (W_k, X_k, Y_k)$ are

$$\boxed{\begin{aligned} W_k &= \sum_{\substack{0 \le i \le p-1 \\ 0 \le j \le p \\ i+j=k}} \lambda_{i,j}^{(k)} s_i, \\ (X_k, Y_k) &= \sum_{\substack{0 \le i \le p-1 \\ 0 \le j \le p \\ i+j=k}} \lambda_{i,j}^{(k)} \left(s_i C_j + d R_L(V_i) \right). \end{aligned}}$$

The rational control point is $Q_k = (X_k/W_k, Y_k/W_k)$ with weight W_k. The second term is degree-elevated by multiplication with the degree-p partition of unity. These finite sums are the local Bernstein form of Albrecht et al., equations (39)-(49), and of the polynomial formula in Farouki, Section 17.5. They are exact coefficient products, not quadrature.

The formula is language- and coordinate-system independent, but C_j, V_i, s_i, and d SHALL use one affine frame. The reference Python/NumPy profile performs these products in the global normalized frame of Section 9.1 with $\widehat{d} = d/H_x$, converts each verified Euclidean rational control back to user coordinates, and leaves the dimensionless weights unchanged. Mixing a user-coordinate distance with normalized span controls is nonconforming.

For the default simple logical knots, the compact Albrecht representation has degree $4m + 1$, clamped endpoint multiplicity $4m + 2$, and internal multiplicity $3m + 2$. The reference repository SHALL instead extract its already-verified compiled Bernstein spans and use the canonical segmented refinement below. This is exact knot insertion into that compact representation. It is not a different curve and does not require a second global Gramian product solve.

15.6.4 Positive-weight refinement

A strictly positive polynomial can have a nonpositive Bernstein coefficient on a wide interval. The public handle requires strictly positive weights for stable standard NURBS interchange. For every homogeneous degree-q patch:

$$\gamma_n = \frac{n\epsilon}{1 - n\epsilon}, \qquad \tau_W = \gamma_{32(q+1)} \max_k |W_k|,$$

where ϵ is the arithmetic unit roundoff. The reference binary64 profile permits at most 24 midpoint subdivisions per source patch. It SHALL:

1. accept a leaf only when every $W_k > \tau_W$;
2. if any coefficient fails, subdivide all three homogeneous Bernstein coordinates by de Casteljau at the exact local midpoint $1/2$;
3. recurse in left-child-before-right-child order;
4. stop only when every leaf is certified positive;
5. raise `OffsetConstructionError` if the existing regularity bound cannot certify termination by level 24.

Since $\sigma > 0$ on a compact certified span, this process terminates in exact arithmetic. Subdivision changes only the representation. It SHALL use the same split for every distance because the denominator is independent of d. An implementation in another arithmetic MAY use a different fixed depth and a proved tighter rounding bound, but it SHALL document both and retain the same accept-or-fail rule. An unresolved weight sign is never accepted.

15.6.5 Canonical NURBS assembly

Order every final rational patch by source traversal. At a connected common endpoint, multiply the next patch by the unique positive projective scale that makes both endpoint homogeneous triples equal, then omit its duplicate first control. The Euclidean endpoints and, for $d \neq 0$, the one-sided normals SHALL be independently verified before this rescale. Projective scaling MUST NOT conceal a geometric mismatch. A nonrepresentable scale raises `OffsetConstructionError`.

Let the M final degree-q patches have exact normalized global breakpoints

$$0 = \xi_0 < \xi_1 < \cdots < \xi_M = 1.$$

The canonical clamped knot vector is

$$\boxed{U = \{\xi_0^{[q+1]}, \xi_1^{[q]}, \ldots, \xi_{M-1}^{[q]}, \xi_M^{[q+1]}\},}$$

where $a^{[r]}$ denotes r consecutive copies of a. Therefore

$$\text{num\_control\_points} = Mq + 1, \qquad \text{num\_spans} = M.$$

Every original compiled-span boundary and every deterministic positivity split is retained at its exact public parameter. Knot removal, degree reduction, and distance-dependent topology are prohibited. The segmented form is intentionally simple and maps directly to the existing immutable `SpanKernel` data. Identical source state and binary64 distance SHALL produce bitwise-identical arrays in the reference Python environment.

For a closed source, the returned NURBS is clamped over one period $[0, 1]$. Its endpoint values coincide, `closed` is true, and no extended periodic or antiperiodic preimage controls are exposed. The hidden preimage sign η cancels from w^2, $|w|^2$, and the offset.

15.6.6 NURBS evaluation

`NURBSHandle.point(u)` SHALL evaluate the homogeneous controls

$$(W_i Q_{i,x}, W_i Q_{i,y}, W_i)$$

with the standard de Boor algorithm and divide once after checking that the denominator is finite and positive. It uses the source spline's unchanged normalized parameter, the same few-ULP endpoint clamp, right-sided internal knot selection, and exact endpoint behavior. It SHALL NOT call back into the source spline.

15.6.7 Verification and atomicity

Offset construction is a verified on-demand compilation. Before publication, an independent path SHALL check:

1. degree, control count, knot count, nondecreasing knots, and clamped endpoint multiplicities;
2. finite Euclidean controls and strictly positive finite weights;
3. source derivative controls against hw^2 and speed controls against $h|w|^2$;
4. every homogeneous Bernstein coefficient of $\sigma z + dR_L(v)$ and σ;
5. both incident values at every source and refinement breakpoint;
6. deterministic interior oracle values against $z(u) + dN_L(u)$ with a degree- and scale-aware binary64 bound;
7. equal seam points for a closed source.

A mutable source SHALL capture one committed version before compilation. The returned handle remains unchanged after later source edits. Any failure raises `OffsetConstructionError` or the more specific validation/continuity error; no partial handle is published.

15.6.8 Cusps and self-intersections

With source arc length s,

$$\frac{dz_d}{ds} = (1 - d\kappa)T.$$

The exact offset has a cusp where $1 - d\kappa = 0$ and can self-intersect for large $|d|$. These are not construction failures and SHALL NOT be trimmed, smoothed, joined, or rejected. A cusp is not a denominator pole: the NURBS denominator is the strictly positive source speed. Cusp classification, trimming, and planar-region Boolean operations are outside the minimal `NURBSHandle` interface.

16. Numerical safety requirements

16.1 General prohibitions

Production code MUST NOT:

- trust a nonlinear optimizer's success flag without independent checks;
- use unbounded loops;
- use random starts by default;
- use finite differences for accepted continuity jets;
- use finite differences for any public derivative, curvature derivative, or curvature-vector query;
- use generic polynomial root solvers in the distance hot path;
- use numerical quadrature for authoritative arc length;
- subtract nearly equal forward cumulative lengths near a span endpoint;
- square unscaled values that may overflow;
- accept NaN or infinity through a clamp;
- silently reduce continuity, degree, or regularity requirements;
- silently rebuild globally when strict-local editing was requested;
- compare geometry with one absolute tolerance independent of scale and conditioning.
- form high-order factorials, binomial coefficients, reciprocal span-width powers, or preimage powers without checked scaling.

16.2 Safe norms and differences

Use scaled `hypot` algorithms for vector norms. Coordinate differences SHALL be computed after safe exponent scaling when direct subtraction risks overflow. If the actual displacement is not representable in binary64 user coordinates, raise `NumericalPrecisionError`.

16.3 FMA and compensated arithmetic

Use `math.fsum` or Neumaier compensation for:

- flat length prefixes;
- cumulative parameter weights;
- parameter totals and periodic knot extensions;
- global displacement and length verification.

FMA/error-free product kernels are permitted future hardening, not a current reference-profile dependency.

16.4 Underflow and overflow

Global normalization SHALL keep solver arithmetic away from subnormal and overflow ranges. Every restoration to physical coordinates or units SHALL check finiteness. Per-patch `ldexp` scaling is a future extension, not a current invariant.

The target test range SHALL include coordinate scales from at least `1e-150` through `1e307`, while recognizing that a tiny displacement superimposed on a huge coordinate may be unrepresentable in binary64 and must be rejected rather than fabricated.

16.5 Small-angle formulas

Any construction formula involving

$$\frac{\sin x}{x}, \quad \frac{1-\cos x}{x^2}, \quad \frac{e^{ix}-1}{x}$$

SHALL use stable `sinc`-style series or half-angle forms near zero.

16.6 Determinism

Given identical binary64 inputs, policies, package version, NumPy/SciPy major versions, and execution architecture, construction SHALL be deterministic. Initializer order, square-root signs, seam monodromy, damping order, line search, and exact-tie decisions are specified; construction does not depend on randomness, container iteration order, or an unspecified root ordering from a third-party routine. Thread-count-dependent reductions SHALL be avoided in acceptance-critical code. Different architectures or dependency versions need not be bitwise identical, but they SHALL satisfy the same verified geometric contract and deterministic decision rules.

16.7 High-order geometry evaluation

The reference evaluator uses finite Bernstein-difference and normalized-Taylor recurrences, canonical endpoint jets, the width-ratio guard in Section 10.2, and final finite-value checks. Join-side `auto` uses the verified degree/order-dependent continuity bound. It does not yet propagate a separate componentwise forward-error object for each runtime jet.

Power-of-two coefficient rescaling, error-free complex products, and componentwise recurrence bounds are recommended future hardening. An implementation that adds them SHALL preserve the elementary identities and exact endpoint/canonical-jet semantics; it MUST NOT change accepted geometry silently.

The evaluator SHALL return only finite binary64 components. If a mathematically nonzero component is outside the representable range, the reciprocal-speed recurrence loses certification, or the propagated uncertainty cannot distinguish two discontinuous one-sided values, it raises `NumericalPrecisionError` or `DiscontinuousDerivativeError` as applicable. It SHALL never return NaN, infinity, a silently saturated value, or a fabricated zero. Exact structural zeros and polynomial derivatives above degree are returned as exact zero vectors.

17. Verification pipeline

17.1 Independent verification principle

Construction and verification SHALL use at least partially independent code paths. Reusing the exact same residual function and declaring it verified is insufficient.

Recommended independent pairs:

- solver displacement in B-spline/extraction form; verifier displacement from compiled Bezier antiderivative controls;
- solver continuity from shared B-spline controls; verifier continuity from left/right local jets;
- speed coefficients from Bernstein products; verifier speed from direct preimage evaluation at certified points plus coefficient identity;
- inverse LUT monotonicity from direct arc samples; runtime acceptance from direct forward/reverse Bernstein arc evaluation.

17.2 Required constructor checks

Before publication, verify:

1. every input point is reached at its interpolation knot within the position bound;
2. every displacement constraint passes the solver residual and the compiled endpoint-displacement residual;
3. every compiled span has been constructively derived from w^2 and $|w|^2$;
4. physical preimage jets through order $m-1$ agree at every ordinary join and agree with sign η at the closed seam, thereby establishing the reported C/G/curvature guarantees;
5. every span is regular with the required margin;
6. every span length is positive and finite;
7. normalized and physical flat length prefixes are finite and strictly increasing and end at their compensated totals;
8. every inverse LUT is strictly monotone with exact endpoint entries;
9. all public endpoint values are finite;
10. closed curves pass seam position, tangent/frame, sign-aware preimage-jet, and rotational/cyclic-equivariance checks.

Representative inverse residuals and equivariance are suite-level acceptance tests rather than work repeated inside every constructor.

17.3 Position tolerance

In normalized coordinates, the reference position bound is

$$\tau_P = f_P \epsilon \max\left(1, \max_i |\widehat{P}_i|\right)(m+1)^2, \qquad f_P = 256.$$

The physical diagnostic bound is $H\tau_P$. The solver targets one quarter of this value; publication requires both the displacement residual and the independently compiled endpoint residual not to exceed τ_P. A fixed absolute tolerance such as `1e-9` is forbidden.

17.4 Continuity tolerance

The reference verifier compares physical preimage derivatives $h^{-q}d^q w/d\nu^q$ after applying the seam sign. Its relative acceptance bound is

$$\tau_J = 4096\epsilon\, 4^m \max(1, m^2).$$

The factor 4^m conservatively covers the two alternating Bernstein endpoint difference ladders. Each residual is divided by $\max(1, |J_-|, |J_+|)$. Implementations MAY use a demonstrably tighter forward-error bound but SHALL retain the sign-aware physical-jet comparison.

The default tangent absolute gate SHALL not exceed `1e-12` for well-conditioned normalized data. Curvature comparison SHALL use relative scaling and default behavior comparable to `1e-10` on unit-scale data.

17.5 Verification report

`BuildDiagnostics` and `EditReport` SHALL include:

- point/span/hidden-span counts and selected preimage degree where applicable;
- solver iterations;
- refinement rounds;
- maximum interpolation and continuity residuals; constructor diagnostics also include their acceptance bounds;
- minimum regularity ratio;
- maximum inverse residual ratio;
- maximum LUT size;
- affected handles/spans, rebuilt/patch span counts, and version transition for edits;
- whether long-double verification was used;

In the reference profile `max_inverse_residual_ratio` is initialized to zero and `longdouble_verification_used` is false; those fields reserve space for future constructor-time inverse sampling and extended-precision verification.

18. Exceptions

18.1 Hierarchy

All package exceptions SHALL derive from `PHSplineError`. `CubicPHSplineError` and `PHBSplineError` SHALL be sibling family roots; neither SHALL inherit from the other. Input/query errors SHALL also derive from `ValueError`; numerical construction/edit/query failures SHALL also derive from `RuntimeError`.

Required hierarchy:

```
PHSplineError
|-- PHSplineValueError, ValueError
|-- PHSplineRuntimeError, RuntimeError
|-- CubicPHSplineError
`-- PHBSplineError

CubicPHSplineValueError
    : CubicPHSplineError, PHSplineValueError
CubicPHSplineRuntimeError
    : CubicPHSplineError, PHSplineRuntimeError
PHBSplineValueError
    : PHBSplineError, PHSplineValueError
PHBSplineRuntimeError
    : PHBSplineError, PHSplineRuntimeError

Shared value errors (both concrete value branches):
    InvalidPointDataError, InsufficientPointDataError,
    NonFiniteCoordinateError, DegeneratePointDataError,
    ParameterOutOfRangeError, ArcLengthOutOfRangeError,
    UndefinedPrincipalNormalError

Shared runtime errors (both concrete runtime branches):
    NonRegularSplineError, ArcLengthInversionError,
    LengthResolutionError, NumericalPrecisionError,
    OffsetConstructionError

PH B-spline-only value errors:
    ContinuitySpecificationError, DiscontinuousDerivativeError,
    StaleHandleError, StaleLocationError
```

```
PH B-spline-only runtime errors:
    ConstructionConvergenceError, InterpolationVerificationError,
    ContinuityVerificationError, LocalEditFailure,
    ResourceLimitError, TransactionError
```

18.2 Structured fields

Every package exception SHALL accept and expose, as applicable:

```
index: int | tuple[int, ...] | None
point_id: int | None
span_id: int | None
operation: str | None
patch: tuple[int, int] | None
quantity: str | None
value: object
bound: object
iteration: int | None
distance: float | None
refinement_depth: int | None
```

The formatted message SHALL include populated fields. Original fields remain machine-readable. An `OffsetConstructionError` SHALL set `operation="offset"`, `distance`, the source `span_id` when known, and `quantity`, `value`, and `bound`. It SHALL set `refinement_depth` for a positivity-refinement failure.

19. Complexity requirements

Let:

- N be user point count;
- M be compiled PH span count including hidden spans;
- R be final rational offset span count after positive-weight refinement;
- m be preimage degree;
- K be edited patch span count;
- I be bounded nonlinear iterations.

For fixed m, bounded hidden-span ratio, and bounded I:

Operation	Required complexity
Initial validation and guide	`O(N)`
Initial construction	`O(N)` per bounded iteration; overall `O(N)` under caps
Scalar `point(u)`	`O(log M + C_eval(m))`
Scalar `arc_length(u)`	`O(log M + C_eval(m))`
Random `point_at_length(s)`	`O(log M + C_eval(m)*I_inv)` with bounded `I_inv`
Sorted batch distance queries	`O(M + Q*C_eval(m))` or `O(Q*C_eval(m))` after the starting span
Local geometry solve/compilation only	`O(K*m^3*I)`, with order-derived bounded `K`
Total strict-local move/insert/delete	`O(N + K*m^3*I)` in the reference flat-prefix profile
Flat metric-prefix rebuild	`O(M)`
Snapshot build	`O(1)` shared state
Sequential cursor advance	amortized `O(1 + crossed_spans)`
Exact NURBS offset build	`O((M + R)*m^2)` time and `O(R*m)` output storage
NURBS `point(u)`	`O(log R + m^2)` for binary search and degree-$(4m+1)$ de Boor evaluation

Absolute timing targets SHALL be benchmark gates, not API guarantees. The local nonlinear portion and total

edit latency SHALL be reported separately; the latter currently includes linear validation and prefix publication. A future augmented-tree profile may target:

- mutable scalar random `point_at_length`: median below 15 microseconds for 100 to 100,000 spans;
- snapshot scalar random `point_at_length`: median below 8 microseconds where Python dispatch dominates;
- one-point strict-local geometry reconstruction approximately independent of total `N`, with total reference-profile latency reported separately;
- no more than two Newton corrections for at least 99.9 percent of benchmark distance queries; all remaining queries use the bounded bisection fallback.

These numbers SHALL be measured on a declared platform and MUST NOT be presented as universal guarantees.

20. Serialization

Future extension. The reference profile does not yet expose persistence; the requirements in this section define a compatible future format and are not current API claims.

20.1 Authoritative state

Serialization SHALL include:

- package format version;
- user points and stable point IDs;
- closed/open flag;
- continuity request and policies;
- knot topology and hidden-span mapping;
- preimage coefficients or authoritative local span representation;
- verification metadata;
- optional compiled inverse kernels and tree aggregates.

20.2 Loading

On load, the implementation SHALL validate schema, finiteness, array sizes, and checksums. By default it SHALL perform lightweight structural and metric verification before exposing the object. A `verify="full"` mode SHALL rerun all postconditions.

Untrusted serialized data MUST NOT be allowed to bypass regularity or bounds checks.

21. Threading and concurrency

Future extension. Immutable snapshot queries are isolated from later edits, but the reference mutable object does not yet claim a multi-writer or reader/writer locking contract. The following requirements apply when such a profile is implemented.

Each mutable concrete PH B-spline SHALL support multiple concurrent readers and a single writer through a read/write lock or immutable-root publication.

- Queries observe one complete version.
- An edit constructs a private candidate and publishes it atomically.
- Query methods SHALL NOT observe partially updated length aggregates or span caches.
- `PHBSplineSnapshot` SHALL be lock-free for reads after construction.
- User callbacks SHALL not be executed while internal locks are held.

22. Testing specification

22.1 Baseline test migration

The new suite SHALL port the intent of every `cubic-ph-spline` test category:

- arc-length evaluation and inversion;
- classification and nonconvex geometry;
- extreme scales;
- frames and normals;
- continuity;
- geometry and endpoint interpolation;
- invariants;
- straight-line cases;
- validation and exceptions.

The existing retrieved baseline passed 491 tests; the new implementation SHALL not reduce adversarial coverage merely because the representation changed.

22.2 Input geometry matrix

Tests SHALL include:

- random convex polygons;
- highly nonconvex open polylines;
- many alternating inflections;
- self-intersections and figure-eight data;
- exact and near 180-degree reversals;
- repeated nonconsecutive points;
- open and closed loops;
- long straight runs mixed with tight turns;
- chord ratios down to the representability/regularity guard;
- 100,000-point data streams.

For any configuration that cannot be constructed under policy limits, the test SHALL verify the exact exception type, structured fields, and transactional rollback.

22.3 Continuity matrix

At minimum test:

- `G0` through `G5`;
- `C0` through `C5`;
- curvature `C0` through `C3`;
- mixed requests such as `G2+C1`, `C3+curvature C1`, and `G4+curvature C2`;
- open boundaries and closed seam continuity.

Verify left/right jets independently in normalized and physical coordinates.

22.3.1 Mandatory closed-seam regression matrix

Closed tests SHALL exercise both the observable curve and the hidden square-root topology:

- periodic and antiperiodic guide winners, with `seam_sign` restricted to exactly ± 1;
- twisted extraction for degrees smaller than, equal to, and larger than the closed base-control count, including multiple wraps of one stencil;
- unwrapped Greville abscissae with the factor η^q;
- sign-aware canonical preimage jets through the requested order;
- exact position closure plus tangent, curvature-vector, and requested derivative continuity at $u = 0/1$;
- cyclic reindexing, rotation, reflection, and rotational-symmetry equivariance;
- wrapped local move, insert, and delete patches at and adjacent to the seam for at least G2 and G8;
- bitwise identity of every exterior span after a wrapped local edit.

The required documentary counterexample is the 24-anchor radial star with alternating radii and 12-fold rotational symmetry at G8. Forcing a periodic preimage ($\eta = +1$) produced a seam-localized loop and destroyed the expected rotational symmetry even though point closure could still pass. Adding only antiperiodic extraction was also insufficient: reducing negative/unwrapped Greville abscissae modulo the period without η^q produced a different

seam-localized multi-sector distortion. The complete correction is the single monodromy invariant propagated through branch selection, extraction, Greville seeding, canonical jets, verification, and local-edit gauge transport.

22.4 Dynamic-edit property tests

For point counts including 10, 1,000, and 100,000:

- move interior and endpoint points;
- insert before, after, and around flat-prefix/span-index boundaries;
- delete interior and endpoint points;
- perform long randomized edit sequences;
- batch edits in transactions;
- induce deliberate strict-local failure and verify exact rollback;
- compare an expanded/global rebuild with a fresh constructor build under the same deterministic policy;
- verify unchanged spans outside the patch are the identical objects when bitwise preservation is enabled;
- verify all retained point handles remain valid and unchanged;
- verify deleted handles become stale only after commit.

22.5 Distance inversion tests

For every tested span, query:

- `0`, `L`, and the midpoint;
- one and several ULPs from both endpoints;
- every LUT node and cell midpoint;
- random uniform distances;
- distances concentrated where speed is minimal;
- spans at the regularity-ratio threshold;
- alternating tiny and huge span lengths;
- targets exactly at flat-prefix boundaries;
- scalar float and `LengthCoordinate` forms;
- sorted and unsorted batch forms.

Acceptance SHALL verify:

$$|S(\nu) - s| \le \tau_s,$$

monotonicity, bracket preservation, and absence of warnings/NaNs.

22.6 Arbitrary-order geometry queries

For curve, scalar-curvature, and curvature-vector parameter derivatives, test every order from zero through at least 12 and selected orders through the configured default limit. Include endpoints, exact and near joins, inflections, certified straight spans, minimum-speed locations, alternating coefficient scales, and orders above the polynomial degree. Verify `side="auto"`, explicit one-sided values, and the required discontinuity exception at joins whose available continuity is too low.

Check the identities

$$T = \frac{D_u z}{\|D_u z\|}, \qquad C_0 = \kappa N_L = D_s^2 z$$

against independently evaluated left/right jets and high-precision references. Compare the single-order methods with their full-jet counterparts, require exact structural zeros where specified, and exercise invalid types, negative orders, resource-limit rejection, and output overflow/underflow. Finite-difference comparisons MAY be used as low-precision smoke tests only; they are not acceptance oracles.

22.7 High-precision oracle tests

An optional test dependency such as `mpmath` SHALL evaluate selected construction, arc-length, derivative-vector, scalar-curvature, curvature-vector, and inverse cases at 100 or more decimal digits. Production code SHALL not depend on this package.

22.8 Scale and transformation invariance

Apply translations, rotations, reflections, and power-of-two scalings across at least `1e-150` through `1e307` where the transformed coordinates remain representable.

Verify:

- positions transform correctly;
- tangents rotate/reflect correctly;
- parameter derivative vectors transform with the spatial scale and the corresponding orthogonal vector transform;
- lengths scale by absolute scale;
- curvature scales inversely;
- every parameter derivative of the curvature vector scales inversely with spatial scale and with the corresponding orthogonal vector transform;
- distance inversion returns corresponding locations;
- edit locality and handle identity remain unchanged.

22.9 Fuzzing

Use property-based testing where practical. Fuzz:

- point arrays and malformed inputs;
- continuity values;
- edit sequences;
- flat-prefix exact boundaries and structural-sharing edges;
- extreme distance values;
- derivative orders and join-side selections;
- LUT sizes;
- serialized data corruption.

No fuzz input may produce an unhandled `RuntimeWarning`, segmentation fault, infinite loop, or untyped exception.

22.10 Performance regression tests

Benchmarks SHALL measure:

- construction time versus `N` at fixed order;
- strict-local geometry-rebuild time and total edit time versus `N` at fixed order-derived patch size;
- full validation, tuple assembly, and flat-prefix publication time;
- scalar random distance access versus `N`;
- mutable object versus snapshot queries;
- sorted batch throughput;
- scalar and batch derivative-vector and curvature-vector throughput versus requested order;
- full-jet throughput versus repeated single-order calls;
- correction-count distribution;
- exact NURBS offset construction time and output-control count versus `N` and preimage degree;
- memory per input point and per hidden span.

The edit benchmark SHALL explicitly separate the approximately flat local solve/span-compilation component from the current linear whole-object validation and flat-prefix publication component for 100, 1,000, and 10,000 points. It MUST report the total latency users observe.

22.11 Exact offset NURBS tests

Exercise mutable curves and immutable snapshots for open and closed G2, G4, and G8 constructions, including periodic and antiperiodic closed preimages. Use positive, negative, zero, near-zero, and large finite distances. Required checks are:

- `degree == 4*preimage_degree + 1`;
- `len(knots) == num_control_points + degree + 1`;
- clamped endpoint multiplicity `degree + 1`, internal multiplicity `degree`, and `num_control_points == num_spans*degree + 1`;
- finite nondecreasing knots, finite controls, and strictly positive weights;
- read-only arrays and the absence of every mutation method;
- exact common parameter domain $[0, 1]$;
- independent direct comparison with $z(u) + dN_L(u)$ at endpoints, all source and refinement knots from both sides, Greville points, and dense random interior values;
- $z_{-d}(u) = z(u) - dN_L(u)$ with the declared left-normal convention;
- identical degree, knots, and weights for all distances from one source state, and homogeneous numerators affine in d;
- deterministic repeated construction and positivity refinement;
- a constructed cusp with $1 - d\kappa = 0$ and a large-distance self-intersection, neither trimmed nor rejected;
- rejection of a nonzero offset at a G0 tangent discontinuity and acceptance of its zero offset;
- atomic version capture: an offset handle remains bitwise unchanged after the mutable source is edited;
- malformed distance arguments, nonrepresentable homogeneous data, and forced positivity-refinement resource exhaustion;
- direct 100-or-more-decimal-digit verification of the homogeneous Bernstein coefficient identities for selected spans.

Sampled point agreement alone is insufficient. The oracle SHALL also compare the coefficient products for $\sigma z + dR_L(v)$ and σ and evaluate the NURBS through an implementation independent of the production evaluator.

23. Acceptance criteria

Version 1 is releasable only when all of the following are true:

1. The documented public API and exception hierarchy are implemented and typed.
2. Default construction of representative arbitrary point data produces a verified regular `G2` spline.
3. All accepted curves satisfy exact point interpolation within the scale-aware bound.
4. Requested finite continuity is verified at every join; closed curves also pass the monodromy, twisted-extraction, Greville-sign, seam-gauge, and symmetry regression matrix in Section 22.3.1.
5. Arbitrary-order derivative-vector and curvature-vector queries use the specified elementary kernels, satisfy their jet identities and error bounds, and never use finite differences.
6. Arc length is generated analytically from PH coefficients with no quadrature.
7. Every scalar inverse query either meets the residual bound or raises a typed exception.
8. Every representable finite signed offset of a globally G1 source returns the exact verified read-only NURBS specified in Section 15.6; no sampled or fitted substitute is present in the production path.
9. Strict-local edits are atomic and leave exterior span kernels bitwise unchanged; benchmarks separately report bounded local reconstruction and current total $O(N)$ publication cost.
10. Reference `expand` admission and explicit global-repair behavior are documented and tested without claiming geometric retry expansion.
11. 100,000-point construction, query, and local-edit tests pass within declared memory limits.
12. Scale, reversal, inflection, self-intersection, and near-regularity adversarial tests pass or fail with the declared typed exceptions.
13. The package emits no `RuntimeWarning` under the test suite.
14. Documentation does not claim universal existence, exact symbolic high-degree inversion, or unconditional successful `O(log N)` edits.

24. Implementation sequence

The recommended work breakdown is:

Phase 1 - immutable mathematical core

- minimum-degree rule and simple-midpoint knot topology;
- open extraction and twisted periodic/antiperiodic closed extraction from endpoint basis derivatives;
- unwrapped sign-aware Greville initialization and canonical shared jets;
- exact speed and arc polynomial generation;
- arbitrary-order derivative and curvature-vector kernels;
- geometry evaluation and regularity certificate;
- shared immutable `NURBSHandle` and homogeneous de Boor evaluator;
- immutable `SpanKernel` tests.

Phase 2 - distance kernel

- compensated flat length prefixes;
- forward/reverse arc evaluators;
- LUT construction;
- safeguarded inverse solver;
- scalar and batch distance tests.

Phase 3 - static PH B-spline constructor

- deterministic secant guide and two-state closed monodromy selection;
- analytic displacement constraints and Jacobian;
- minimum-norm sparse constraint projection with deterministic damping;
- direct endpoint-jet extraction and recursive Bernstein-box regularity;
- exact homogeneous offset products, deterministic positive-weight refinement, and canonical NURBS assembly;
- full independent verification;
- immutable snapshot API.

Phase 4 - local geometry edits and flat publication

- stable handles and locations;
- order-derived clamped patch with fixed exterior jets;
- wrapped closed-patch gauge transport;
- transactional move, insertion, and deletion;
- structural sharing of exterior span kernels;
- rebuilt flat parameter/length prefixes and immutable snapshots.

Phase 5 - enterprise hardening

- optional augmented point/span/metric tree;
- extreme-scale local frames;
- serialization;
- concurrency;
- fuzzing and high-precision oracles;
- 100,000-point benchmarks;
- documentation and migration guide from `CubicPHSplineOpen`.

No phase should introduce a public unverified fallback merely to keep the API running.

25. Example usage

```
import numpy as np
```

```python
from ph_spline import PHBSplineOpen

points = np.array(
    [
        [0.0, 0.0],
        [1.0, 0.4],
        [2.0, -0.7],
        [3.0, 1.1],
        [2.2, 2.0],
    ],
    dtype=np.float64,
)

# With no explicit continuity arguments, the guarantee defaults to G2.
curve = PHBSplineOpen(points)

L = curve.length
p = curve.point_at_length(0.37 * L)
u = curve.parameter_at_length(0.37 * L)
t = curve.tangent(u)

# Exact parallel curve: positive distance is to the traversal-left side.
offset = curve.offset(0.125)
assert offset.degree == 4 * curve.preimage_degree + 1
assert offset.domain == (0.0, 1.0)
assert np.allclose(offset.point(u), curve.point(u) + 0.125 * curve.normal(u))

# Elementary arbitrary-order geometry kernels; no numerical differencing.
d5 = curve.derivative(u, 5)
k3 = curve.curvature_vector(u, 3)  # D_u^3(kappa * N_left)
z_jet = curve.jet(u, 6)
k_jet = curve.curvature_vector_jet(u, 4)

# Stable handle survives insertion and index shifts.
h = curve.point_handle(2)
report = curve.move_point(h, [2.1, -0.9], repair="strict_local")

inserted = curve.insert_point(3, [2.7, 0.2])
curve.delete_point(inserted.handle)

# Local relative travel avoids loss of global distance resolution.
loc = curve.location_at_length(0.5 * curve.length)
loc2 = curve.advance_by_length(loc, 1.0e-6)
p2 = curve.point_after_length(loc, 1.0e-6)

# Higher-order request. Degree is deduced, not fixed in the class.
curve_c4 = PHBSplineOpen(points, c_order=4, curvature_order=2)
assert curve_c4.preimage_degree == 4
assert curve_c4.degree == 2 * curve_c4.preimage_degree + 1

# Lock-free query object.
snapshot = curve.snapshot()
positions = snapshot.points_at_length(np.linspace(0.0, snapshot.length, 10000))
```

26. Required pseudocode

26.1 Scalar distance query

```
function location_at_length(s):
    target = validate_length_coordinate(s) / global_scale
    if target == 0: return first span at u=0
    if target == normalized_total_length: return last span at u=1

    span_id = searchsorted(normalized_prefix, target, side="right") - 1
    local_s = target - normalized_prefix[span_id]
    span = spans[span_id]

    reverse = local_s > endpoint_reverse_threshold * span.length
    goal = span.length - local_s if reverse else local_s
    lut_s = reversed_complements(span.lut_s) if reverse else span.lut_s
    cell = searchsorted(lut_s, goal, side="right") - 1
    lo, hi = parameter_bracket(cell, reverse)
    u = linear_parameter_seed(lut_s[cell:cell+2], goal, lo, hi)

    for iteration in 0 .. fast_iterations-1:
        f, fp = fast_power_arc_residual_and_speed(u, goal, reverse)
        proposal = u - f / fp
        if proposal invalid: break
        u = proposal

    for iteration in 0 .. max_iterations-1:
        f = authoritative_bernstein_arc_residual(u, goal, reverse)
        if abs(f) <= 64*eps*span.length + 4*ulp(goal):
            return CurveLocation(span_id, u, version)
        update bracket from sign(f) and direction
        fp = signed_direct_speed_from_preimage(u, reverse)
        proposal = u - f / fp
        if proposal is nonfinite or outside bracket:
            proposal = midpoint(bracket)
        u = proposal

    raise ArcLengthInversionError(...)
```

26.2 Strict-local move

```
function move_point(handle, new_point, repair="strict_local"):
    begin private transaction
    resolve handle and validate new_point
    modify candidate interpolation record
    recompute normalized points and parameter widths in retained global frame

    if repair == "global":
        candidate = full_build(candidate_points)
    else:
        changed = intervals whose stable edge, endpoints, or width changed
        patch = shortest_contiguous_patch(changed, logical_count=m+3)
        require one exterior interval for a closed patch
        freeze physical exterior preimage jets through order m-1
        if patch crosses closed seam:
            transport right boundary jet through seam_sign
        build clamped open midpoint basis and guide in patch
        impose endpoint jets by direct extraction-block solves
```

```
        solve exact displacement constraints on free controls by dense lstsq
        compile patch and transport post-seam preimages back to stored gauge
        structurally share every exterior span
        verify all joins touching rebuilt spans with sign-aware seam rule
        candidate = assemble full span tuple and flat prefixes

    require rebuilt span count <= selected admission limit
    independently verify candidate postconditions
    atomically publish candidate and increment version
    return EditReport
```

27. Design rationale and nonclaims

27.1 Why quintic is the default

A degree-1 complex preimage produces a cubic PH curve whose signed curvature numerator is constant, so a regular nonstraight cubic PH segment cannot contain an inflection. A degree-2 preimage produces a quintic PH curve with enough local shape freedom to support inflections while retaining low-degree speed and arc polynomials. Default `G2` therefore maps naturally to $m = 2$, degree 5.

27.2 Why the inverse is numerical despite polynomial arc length

For a degree-m preimage, the arc polynomial has degree $2m + 1$. Generic quintic and higher polynomials have no universal radical inverse. Restricting every speed polynomial to a radical-invertible compositional family would materially reduce planar shape freedom.

The specified LUT plus safeguarded monotone correction retains the general PH B-spline space while giving predictable, verified machine-precision queries. It is also likely to be faster than evaluating multiple transcendental special functions for many spans.

27.3 Why local support needs patch closure

Compact support of the preimage affects the hodograph locally, but position is its integral. Without a zero net displacement change across an edited patch, all downstream positions would translate. The fixed boundary positions and patch displacement constraints are therefore part of the local-support definition, not an optional refinement.

27.4 Why strict locality can fail

A large point move can make the fixed exterior jets incompatible with any regular PH curve inside a small patch. No implementation can guarantee both successful arbitrary edits and an immutable finite patch in all cases. The API exposes the honest alternatives:

- strict local and transactional failure;
- expanding patch with potentially larger cost;
- explicit global rebuild.

27.5 Why a closed curve can require an antiperiodic preimage

The PH map squares the preimage angle. A tangent making one full turn around a regular simple closed curve lifts to a square root making one half-turn, so the lift returns with the opposite sign. Requiring `w(T) == w(0)` confuses the curve's periodic observable w^2 with a choice of square-root gauge. The correct seam invariant is `w(T) == eta*w(0)` with `eta` selected from both topological possibilities and propagated everywhere the basis wraps.

This distinction remained hidden on ordinary circles and low-order examples because position closure and even several frame checks can pass while the shape defect is concentrated near the chosen seam. The G8 alternating-radius radial star made the error unmistakable: a periodic-only basis broke exact 12-fold equivariance and introduced a local loop.

27.6 Why the seam sign must reach Greville seeds and edits

Twisted extraction alone defines the right function space but does not give a coherent initial control field. Closed Greville abscissae extend outside the base period; wrapping their parameters without wrapping their gauge changes the seed discontinuously. Likewise, a wrapped local patch is an open solve on the universal cover and its boundary jets live in that lifted gauge. The same single `eta` must therefore govern control extension, seed evaluation, canonical jets, verification, and patch transport. Treating any one of these as an unrelated seam exception recreates the defect in a different stage.

Appendix A. Suggested diagnostic dataclasses

```
@dataclass(frozen=True, slots=True)
class ContinuitySpec:
    g_order: int | None
    c_order: int | None
    curvature_order: int | None

@dataclass(frozen=True, slots=True)
class BuildDiagnostics:
    point_count: int
    span_count: int
    hidden_span_count: int
    preimage_degree: int
    iterations: int
    refinement_rounds: int
    max_interpolation_residual: float
    interpolation_bound: float
    max_continuity_residual: float
    continuity_bound: float
    min_regularity_ratio: float
    max_inverse_residual_ratio: float
    max_lut_nodes: int
    longdouble_verification_used: bool

@dataclass(frozen=True, slots=True)
class EditReport:
    operation: str
    version_before: int
    version_after: int
    affected_point_ids: tuple[int, ...]
    affected_span_ids: tuple[int, ...]
    rebuilt_span_count: int
    patch_span_count: int
    iterations: int
    refinement_rounds: int
    hidden_spans_added: int
    max_interpolation_residual: float
    max_continuity_residual: float
    min_regularity_ratio: float

@dataclass(frozen=True, slots=True)
class InsertResult:
    handle: PointHandle
    report: EditReport

@dataclass(frozen=True, slots=True)
class Frame2D:
```

```
    point: NDArray[np.float64]
    tangent: NDArray[np.float64]
    left_normal: NDArray[np.float64]
    signed_curvature: float
```

Appendix B. Double-double primitives

Future extension. The reference profile uses Neumaier-compensated flat prefixes. An augmented double-double length tree SHOULD use standard error-free transformations:

```
TwoSum(a, b):
    s = a + b
    bb = s - a
    e = (a - (s - bb)) + (b - bb)
    return s, e

FastTwoSum(a, b), requiring |a| >= |b|:
    s = a + b
    e = b - (s - a)
    return s, e
```

Double-double addition SHALL renormalize the pair. Tree comparisons SHALL compare the exact pair lexicographically only after accounting for overlapping low parts; a tested helper type is required rather than ad hoc tuple arithmetic.

Appendix C. Formal derivative and curvature-vector jet recurrence

Let $w(t)$ be represented by normalized truncated power-series coefficients around a query or join. Construct series for:

$$A(t) = \operatorname{Im}(\overline{w}w'), \qquad B(t) = |w|^2.$$

Then

$$\kappa(t) = 2A(t)B(t)^{-2}.$$

The curve parameter derivatives follow directly from

$$z'(t) = w(t)^2,$$

and the curvature-vector series follows from the elementary complex expression

$$C_0(t) = 2A(t)\, i\, w(t)^2 B(t)^{-3}.$$

Use finite formal-series multiplication, reciprocal, and differentiation to obtain the parameter derivatives. Compute one reciprocal series for B^{-1} and obtain B^{-2} and B^{-3} by multiplication; do not run independent divisions for each requested order. Use $F = z$, $F = \kappa$, or $F = C_0$ for the curve, scalar-curvature, or curvature-vector parameter jet respectively. The recurrence applies componentwise to vector fields. Normalized Taylor coefficients SHALL be used throughout, and conversion to ordinary derivative values occurs once at the API boundary with checked exponent scaling. This elementary procedure, or an algebraically identical generated recurrence, SHALL be used in production and verification. A general-purpose automatic-differentiation dependency is unnecessary, and numerical differencing is not acceptable.

Appendix D. Benchmark reporting template

Every published benchmark SHALL report:

```
CPU model and clock policy
operating system
Python version
NumPy and SciPy versions
compiler/runtime details
thread environment variables
continuity request and actual degree
number of input points and hidden spans
regularity-ratio distribution
LUT-node distribution
warm-up procedure
number of repetitions and statistic reported
construction time
move/insert/delete latency
random point_at_length latency
sorted batch throughput
correction-count histogram
fallback count
peak resident memory
```

Reference benchmarks SHALL include the mutable object and immutable snapshot, report local reconstruction separately from total flat-prefix edit latency, and identify any future tree profile by name. A query benchmark that omits residual verification does not measure the production contract.

Appendix E. Reconstruction conformance ledger

The following ledger is the short-form guard against repeating the basis and seam mistakes corrected in revision 0.4. A reimplementation is not reference-profile conforming unless every row is true.

Concern	Required reference decision	Prohibited substitution
Continuity degree	$m = \max(2, g, c, k+2)$ and curve degree $2m+1$	raising degree for shape freedom; zeroing invented high derivatives
Knot topology	one simple midpoint knot per user interval; two compiled spans	solving first and refining afterwards; degree inflation
Open basis	clamped, $2N+m$ controls for N intervals	periodic wrapping
Closed basis	$2N$ controls with $c_{j+q(2N)} = \eta^q c_j$	unconditional modulo-periodic controls
Seam selection	minimize both $\eta = +1$ and $\eta = -1$ cyclic root lifts	forcing periodic w because z is closed
Greville seed	keep abscissae unwrapped and apply $\eta^{\lfloor x/T \rfloor}$	modulo reduction without gauge sign
Extraction	direct endpoint basis-derivative jets; twist before wrapped-column collapse	sampled collocation inversion
Shared jets	canonical physical jet, sign $+1$ ordinarily and η at seam	independent cancellation-prone endpoint ladders
Constraints	exact two-subspan Bernstein-Gram displacement and analytic Jacobian	quadrature or finite-difference Jacobian
Shape choice	deterministic minimum-norm projection from guide seed	undocumented strain/elastica/length objective
Regularity	recursive Bernstein bounding boxes and ratio gate	sample-only nonzero checks

Concern	Required reference decision	Prohibited substitution
Local patch	$m+3$ logical intervals, fixed exterior jets through $m-1$	order-independent 7/7/6 folklore
Wrapped edit	solve in lifted open gauge; transport by η on crossing	treating array index zero as an ordinary join
Publication	structurally share exterior spans; rebuild verified flat prefixes	claiming current total edit cost is size-independent
Inverse	parameter-uniform LUT, linear seed, bracketed Newton/bisection	undocumented alternate seed or iteration method
Exact offsets	homogeneous Bernstein products of stored speed and hodograph; deterministic distance-independent positive-weight refinement; verified segmented degree-$4m+1$ NURBS	sampled, interpolated, or fitted offsets; unverified or mutable handles; distance-dependent refinement topology; cusp trimming

At minimum, reconstruction acceptance SHALL include exact `preimage_degree == r_*` tests, G2/G4/G8 interpolation and jet checks, the G8 12-fold radial-star symmetry regression, multiple-wrap twisted extraction, G2/G8 seam-crossing local edits with exterior structural sharing, and the exact-offset matrix of Section 22.11 for open and closed G2/G4/G8 sources.

28. References

1. H. Abraham, `cubic-ph-spline`, version 1.1.0 metadata and repository documentation, reviewed 2026-08-05.
2. G. Albrecht, C. V. Beccari, J.-C. Canonne, and L. Romani, "Pythagorean-Hodograph B-Spline Curves", *Computer Aided Geometric Design* 57 (2017), 57-77, DOI 10.1016/j.cagd.2017.09.001.
3. R. T. Farouki, *Introduction to Pythagorean-Hodograph Curves*, Springer-related author manuscript/material.
4. C. Giannelli, L. Sacco, and A. Sestini, "A local C2 Hermite interpolation scheme with PH quintic splines for 3D data streams", 2021.
5. M. Knez, F. Pelosi, and M. L. Sampoli, "Construction of G2 planar Hermite interpolants with prescribed arc lengths", 2022.
6. J. Kosinka and M. Lavicka, "Pythagorean Hodograph Curves: A Survey of Recent Advances", *Journal for Geometry and Graphics* 18(1), 2014.
7. R. T. Farouki, *Pythagorean-Hodograph Curves: Algebra and Geometry Inseparable*, Springer, 2008, Section 17.5, for the general homogeneous rational-offset formula and the explicit cubic/quintic PH Bezier cases.
8. K. Mørken, "Some identities for products and degree raising of splines", *Constructive Approximation* 7 (1991), 195-208, for exact B-spline products and degree elevation.
9. X. Che, G. Farin, Z. Gao, and D. Hansford, "The product of two B-spline functions", *Advanced Materials Research* 186 (2011), 445-448, for the unique Gramian product-basis construction used by Albrecht et al.
10. L. Piegl and W. Tiller, *The NURBS Book*, second edition, Springer, 1997, for homogeneous de Boor evaluation, knot insertion, and rational Bezier decomposition.